Assignment I - Smart Energy Grid Analytics

Group members - Thierry SEBAKUNZI (101394) - Kwezi James (101502) - Nishyirimbere Yves (101388)

Course: MSDA9215 Advanced Big Data Analytics **Instructor:** Temitope Oguntade

GitHub repository (full code, SQL, logs, screenshots): https://github.com/code-gnosis/Group10_Big_Data_Analytics_assignment_1

All screenshots show the macOS menu bar with the clock and the Dock, as the brief asks.

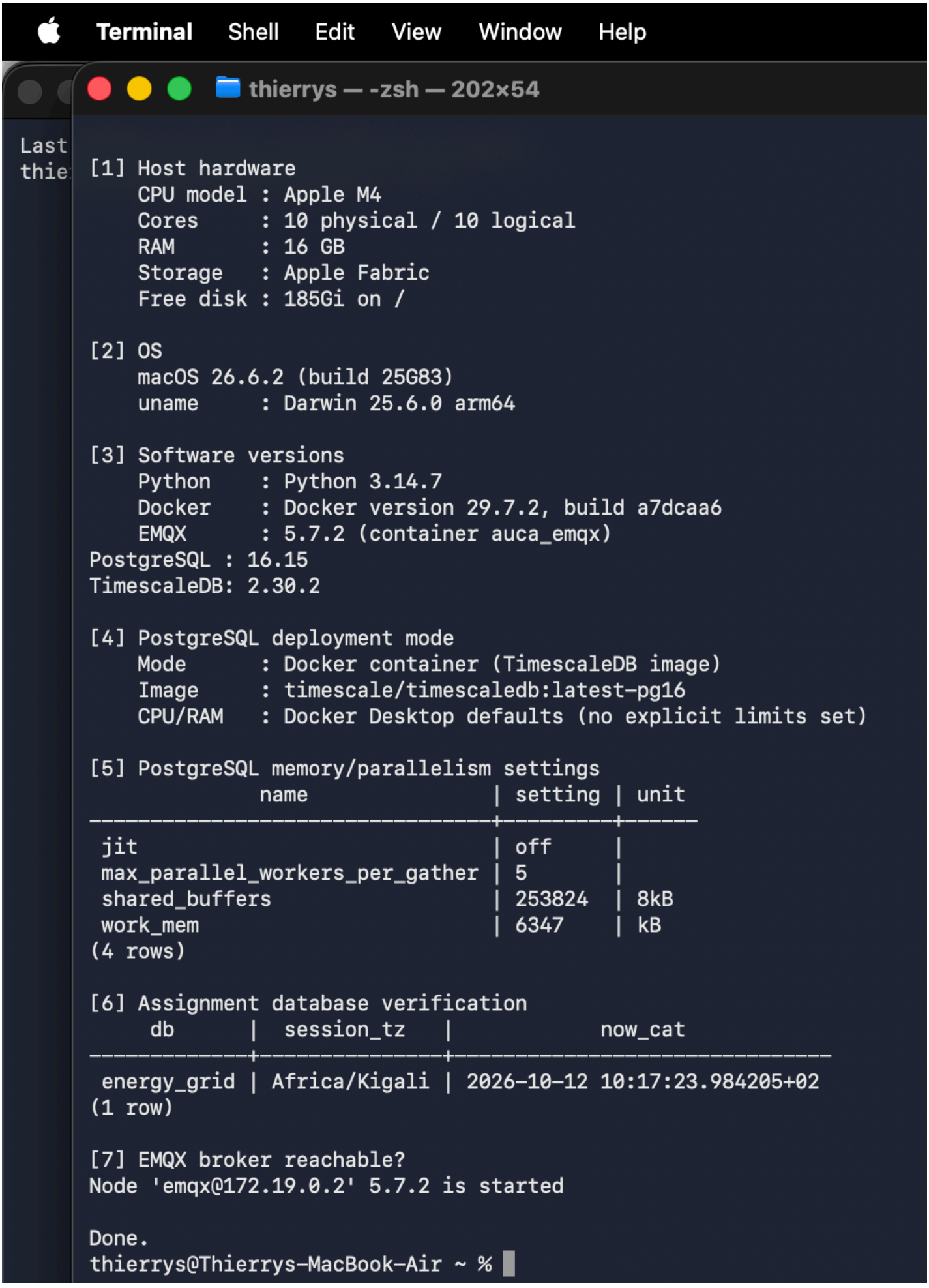
---

1 Build the ingestion pipeline

1.1 Confirm the environment [3 marks]

Item	Value
CPU	Apple M4
Cores	10 physical / 10 logical
RAM	16 GB
OS	macOS 26.6.2 (build 25G83)
Storage	Apple Fabric (NVMe)
Python	3.14.7
EMQX	5.7.2 (Docker container)
PostgreSQL	16.15
TimescaleDB	2.30.2
Deployment	Docker container (no CPU/RAM limits set)
shared_buffers	253824 (2 GB)
work_mem	6347 kB
max_parallel_workers_per_gather	5
jit	off

Database `energy_grid` was created with `TimeZone = Africa/Kigali` so daily buckets align with CAT midnight.


Terminal Shell Edit View Window Help
thierrys — -zsh — 202×54
Last
thie
[1] Host hardware
CPU model : Apple M4
Cores : 10 physical / 10 logical
RAM : 16 GB
Storage : Apple Fabric
Free disk : 185Gi on /
[2] OS
macOS 26.6.2 (build 25G83)
uname : Darwin 25.6.0 arm64
[3] Software versions
Python : Python 3.14.7
Docker : Docker version 29.7.2, build a7dcaa6
EMQX : 5.7.2 (container auca_emqx)
PostgreSQL : 16.15
TimescaleDB: 2.30.2
[4] PostgreSQL deployment mode
Mode : Docker container (TimescaleDB image)
Image : timescale/timescaledb:latest-pg16
CPU/RAM : Docker Desktop defaults (no explicit limits set)
[5] PostgreSQL memory/parallelism settings
name | setting | unit
jit | off |
max_parallel_workers_per_gather | 5 |
shared_buffers | 253824 | 8kB
work_mem | 6347 | kB
(4 rows)
[6] Assignment database verification
db | session_tz | now_cat
energy_grid | Africa/Kigali | 2026-10-12 10:17:23.984205+02
(1 row)
[7] EMQX broker reachable?
Node 'emqx@172.19.0.2' 5.7.2 is started
Done.
thierrys@Thierrys-MacBook-Air ~ %

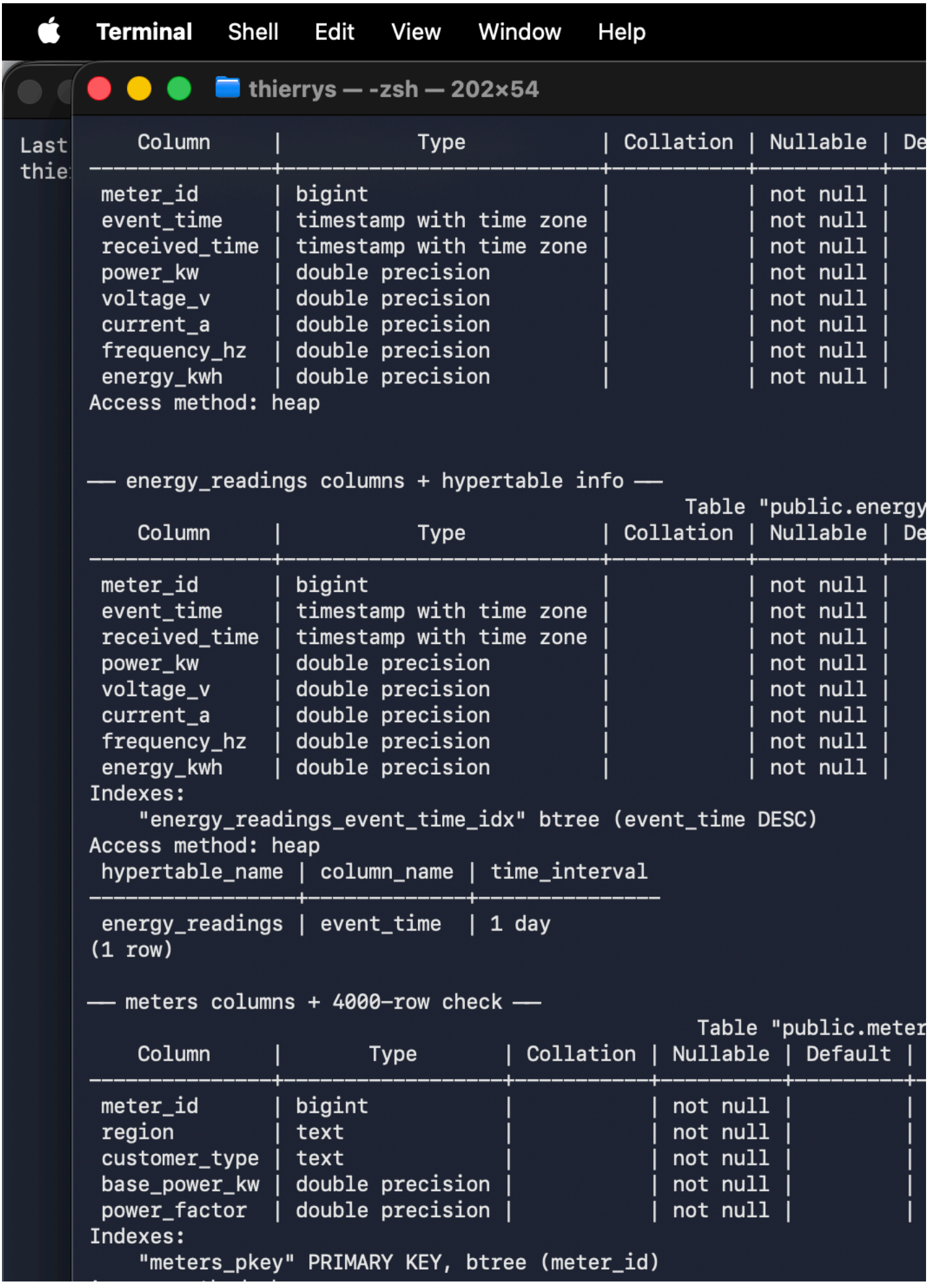
1.1 environment

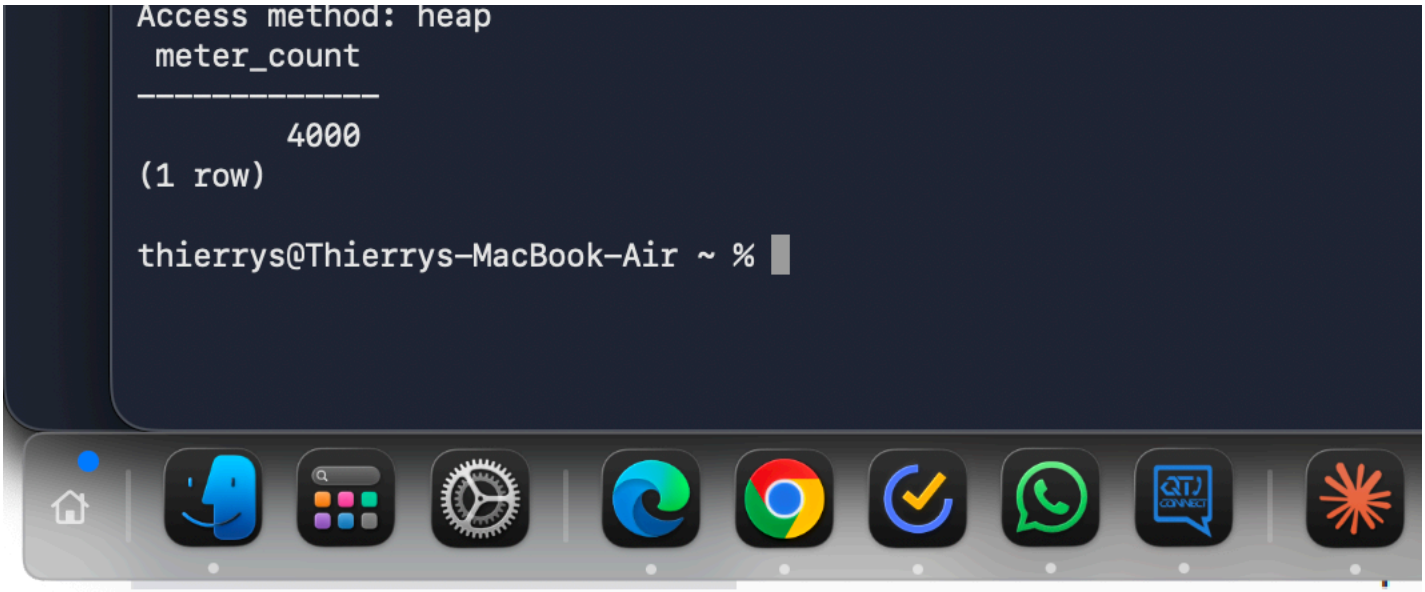
1.2 Create the database structures [3 marks]

Three tables created in `sql/02_schema.sql`: - `energy_live` - regular table for the pilot - `energy_readings` - TimescaleDB hypertable, 1-day chunks - `meters` - meter metadata (meter_id PK)

Both reading tables have the same 8 columns: `meter_id BIGINT`, `event_time TIMESTAMPTZ`, `received_time TIMESTAMPTZ`, `power_kw`, `voltage_v`, `current_a`, `frequency_hz`, `energy_kwh` (all DOUBLE PRECISION).

The loader script `code/02_load_meters.py` calls `meter_metadata()` and inserts the 4,000 rows into `meters`. Count check returns 4000.


Terminal Shell Edit View Window Help
thierrys — -zsh — 202×54
Last
thie
Column | Type | Collation | Nullable | De
meter_id | bigint | | not null |
event_time | timestamp with time zone | | not null |
received_time | timestamp with time zone | | not null |
power_kw | double precision | | not null |
voltage_v | double precision | | not null |
current_a | double precision | | not null |
frequency_hz | double precision | | not null |
energy_kwh | double precision | | not null |
Access method: heap
── energy_readings columns + hypertable info ──
Table "public.energy
Column | Type | Collation | Nullable | De
meter_id | bigint | | not null |
event_time | timestamp with time zone | | not null |
received_time | timestamp with time zone | | not null |
power_kw | double precision | | not null |
voltage_v | double precision | | not null |
current_a | double precision | | not null |
frequency_hz | double precision | | not null |
energy_kwh | double precision | | not null |
Indexes:
"energy_readings_event_time_idx" btree (event_time DESC)
Access method: heap
hypertable_name | column_name | time_interval
energy_readings | event_time | 1 day
(1 row)
── meters columns + 4000-row check ──
Table "public.meter
Column | Type | Collation | Nullable | Default |
meter_id | bigint | | not null |
region | text | | not null |
customer_type | text | | not null |
base_power_kw | double precision | | not null |
power_factor | double precision | | not null |
Indexes:
"meters_pkey" PRIMARY KEY, btree (meter_id)



1.2 tables and meter count

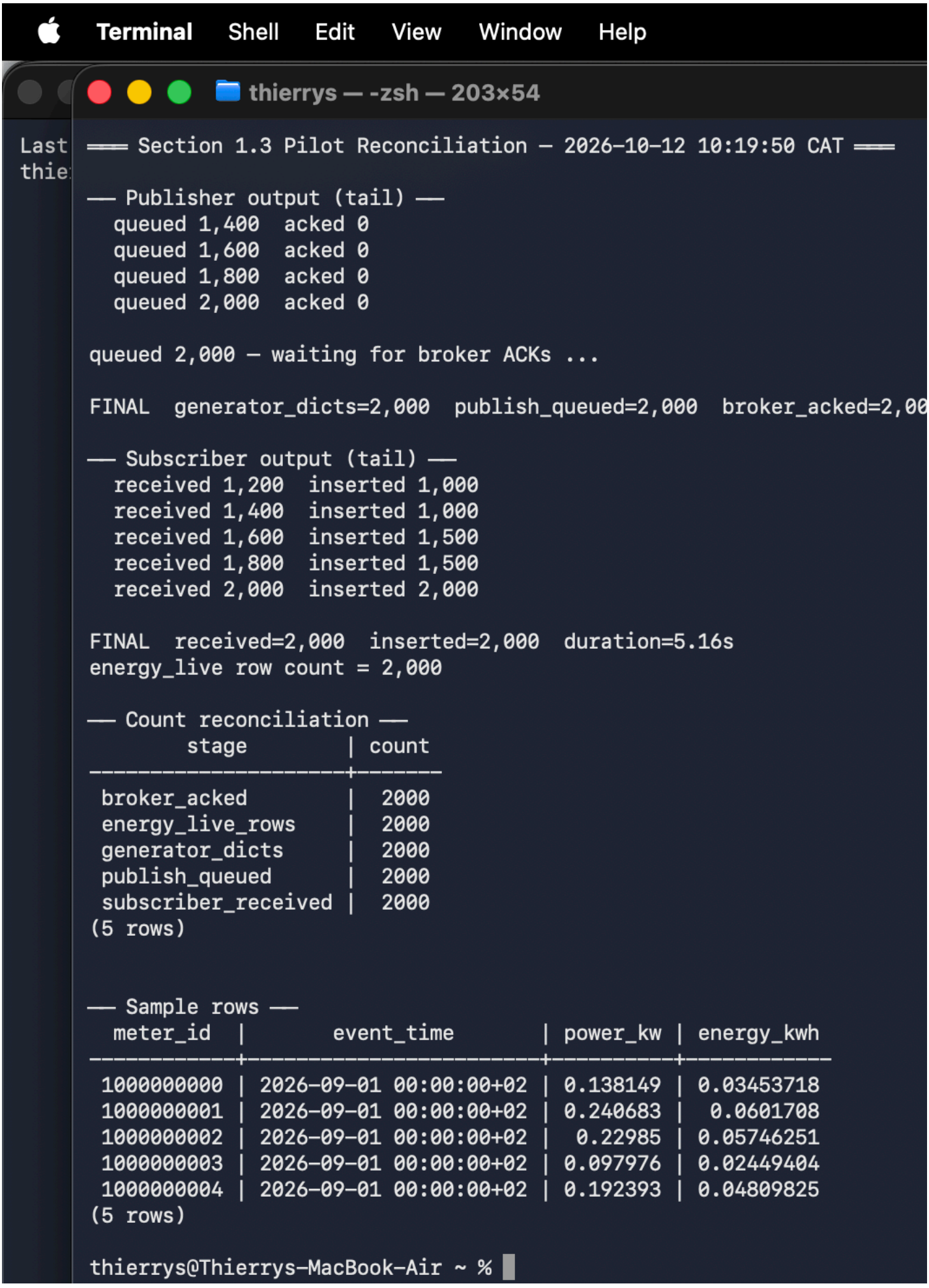
1.3 Capture and publish a pilot stream [4 marks]

Publisher: takes the first 2,000 dictionaries from `iter_readings()` and publishes JSON to `energy/meters/{meter_id}` at QoS 1.

Subscriber: subscribes to `energy/meters/#` at QoS 1 and inserts into `energy_live`.

Reconciliation (all four counts match)

Stage	Count
generator dictionaries	2000
publish queued	2000
broker ACKs	2000
subscriber received	2000
rows in energy_live	**2000**


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
=== Section 1.3 Pilot Reconciliation — 2026-10-12 10:19:50 CAT ===
── Publisher output (tail) ──
queued 1,400 acked 0
queued 1,600 acked 0
queued 1,800 acked 0
queued 2,000 acked 0
queued 2,000 — waiting for broker ACKs ...
FINAL generator_dicts=2,000 publish_queued=2,000 broker_acked=2,00
── Subscriber output (tail) ──
received 1,200 inserted 1,000
received 1,400 inserted 1,000
received 1,600 inserted 1,500
received 1,800 inserted 1,500
received 2,000 inserted 2,000
FINAL received=2,000 inserted=2,000 duration=5.16s
energy_live row count = 2,000
── Count reconciliation ──
stage | count
broker_acked | 2000
energy_live_rows | 2000
generator_dicts | 2000
publish_queued | 2000
subscriber_received | 2000
(5 rows)
── Sample rows ──
meter_id | event_time | power_kw | energy_kwh
1000000000 | 2026-09-01 00:00:00+02 | 0.138149 | 0.03453718
1000000001 | 2026-09-01 00:00:00+02 | 0.240683 | 0.0601708
1000000002 | 2026-09-01 00:00:00+02 | 0.22985 | 0.05746251
1000000003 | 2026-09-01 00:00:00+02 | 0.097976 | 0.02449404
1000000004 | 2026-09-01 00:00:00+02 | 0.192393 | 0.04809825
(5 rows)
thierrys@Thierrys-MacBook-Air ~ %

1.3 pilot reconciliation

---

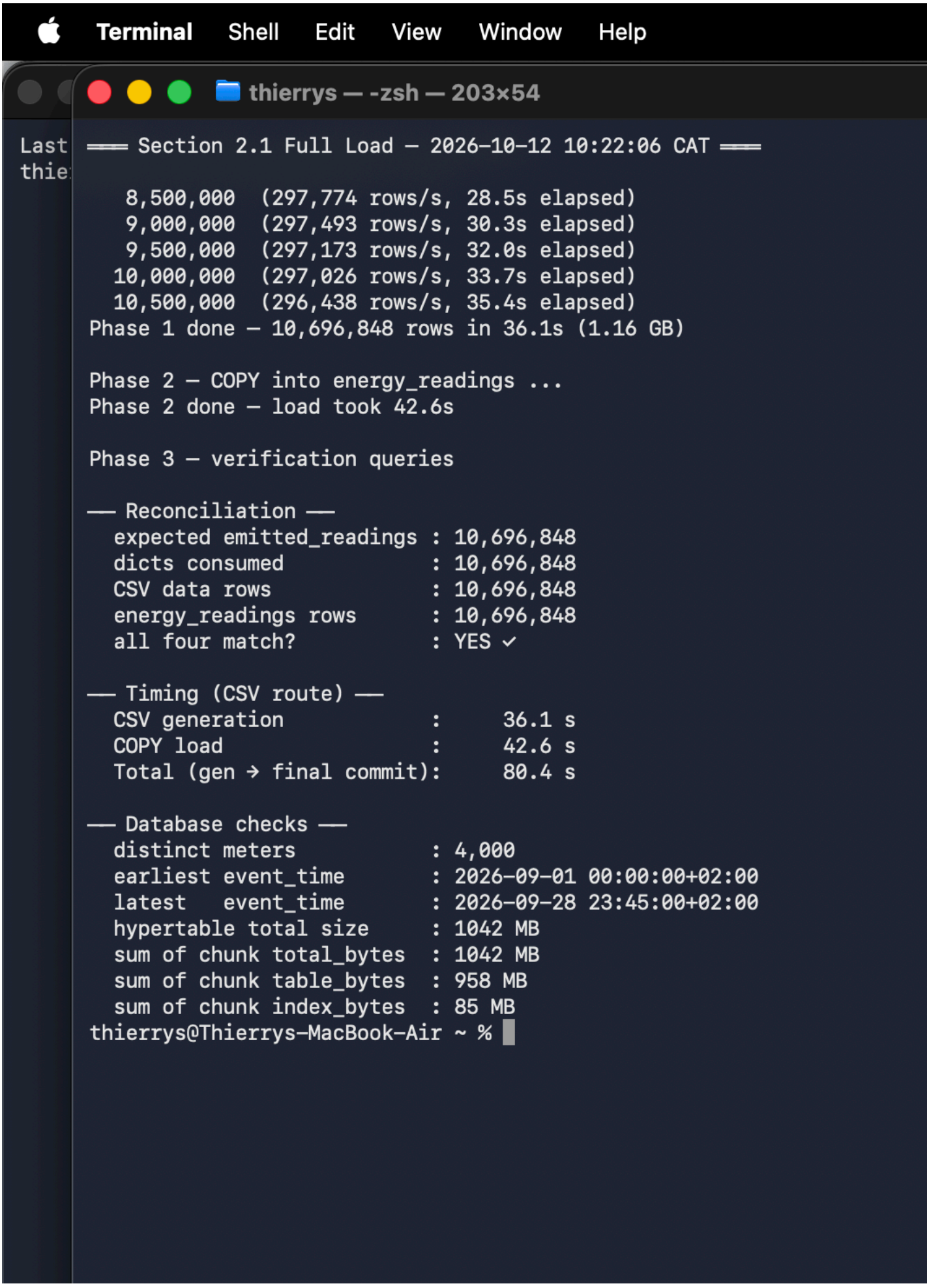
2 Capture and prepare the historical dataset

2.1 Capture and load the complete stream [4 marks]

Chosen route: **CSV + Postgres COPY**. CSV generation and bulk load are much faster than row-by-row inserts for 10M rows.

Metric	Value
expected emitted_readings	10,696,848
dictionaries consumed	10,696,848
CSV rows	10,696,848
rows in energy_readings	10,696,848
CSV size on disk	1.16 GB
CSV generation time	36.1 s
COPY load time	42.6 s
total elapsed	**80.4 s**

Database checks: distinct meters = 4000, earliest = 2026-09-01 00:00+02, latest = 2026-09-28 23:45+02, hypertable total size = 1042 MB (data 958 MB + index 85 MB), 29 chunks.


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
═══ Section 2.1 Full Load — 2026-10-12 10:22:06 CAT ═══
8,500,000 (297,774 rows/s, 28.5s elapsed)
9,000,000 (297,493 rows/s, 30.3s elapsed)
9,500,000 (297,173 rows/s, 32.0s elapsed)
10,000,000 (297,026 rows/s, 33.7s elapsed)
10,500,000 (296,438 rows/s, 35.4s elapsed)
Phase 1 done — 10,696,848 rows in 36.1s (1.16 GB)
Phase 2 — COPY into energy_readings ...
Phase 2 done — load took 42.6s
Phase 3 — verification queries
── Reconciliation ──
expected emitted_readings : 10,696,848
dicts consumed : 10,696,848
CSV data rows : 10,696,848
energy_readings rows : 10,696,848
all four match? : YES ✓
── Timing (CSV route) ──
CSV generation : 36.1 s
COPY load : 42.6 s
Total (gen → final commit): 80.4 s
── Database checks ──
distinct meters : 4,000
earliest event_time : 2026-09-01 00:00:00+02:00
latest event_time : 2026-09-28 23:45:00+02:00
hypertable total size : 1042 MB
sum of chunk total_bytes : 1042 MB
sum of chunk table_bytes : 958 MB
sum of chunk index_bytes : 85 MB
thierrys@Thierrys-MacBook-Air ~ %

2.1 full load

2.2 Investigate data quality [3 marks]

All measured counts match the assignment's reference table exactly.

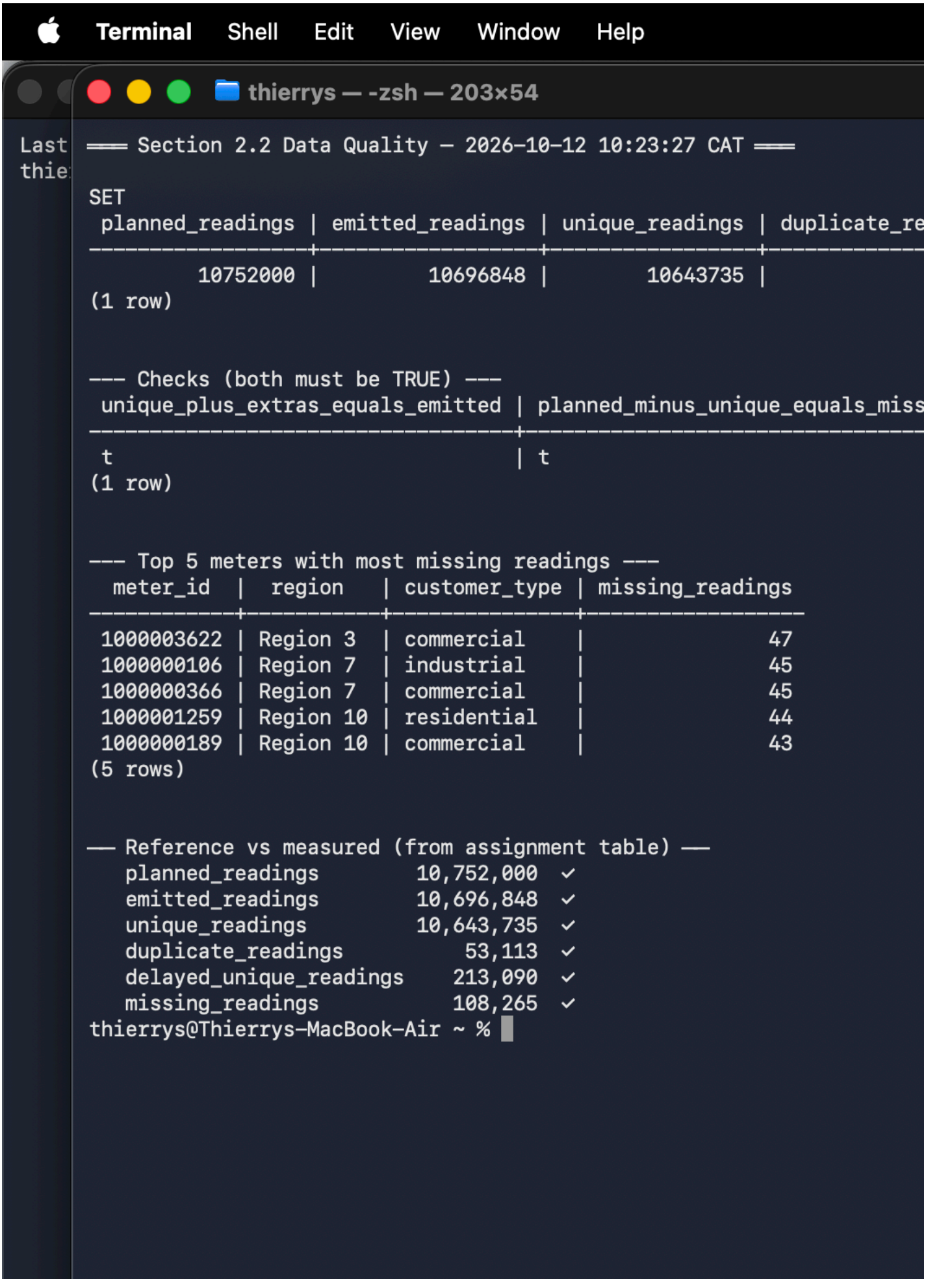
Entry	Expected	Measured
planned_readings	10,752,000	10,752,000
emitted_readings	10,696,848	10,696,848
unique_readings	10,643,735	10,643,735
duplicate_readings	53,113	53,113
delayed_unique_readings	213,090	213,090
missing_readings	108,265	108,265

Percentages: missing 1.0069%, duplicate 0.4990%, delayed 2.0020%, completeness 98.9931%.

Internal checks pass: unique + extras = emitted; planned - unique = missing.

Top 5 meters with most missing readings

meter_id	region	customer_type	missing
1000003622	Region 3	commercial	47
1000000106	Region 7	industrial	45
1000000366	Region 7	commercial	45
1000001259	Region 10	residential	44
1000000189	Region 10	commercial	43


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
=== Section 2.2 Data Quality — 2026-10-12 10:23:27 CAT ===
SET
planned_readings | emitted_readings | unique_readings | duplicate_re
10752000 | 10696848 | 10643735 |
(1 row)
--- Checks (both must be TRUE) ---
unique_plus_extras_equals_emitted | planned_minus_unique_equals_miss
t | t
(1 row)
--- Top 5 meters with most missing readings ---
meter_id | region | customer_type | missing_readings
1000003622 | Region 3 | commercial | 47
1000000106 | Region 7 | industrial | 45
1000000366 | Region 7 | commercial | 45
1000001259 | Region 10 | residential | 44
1000000189 | Region 10 | commercial | 43
(5 rows)
── Reference vs measured (from assignment table) ──
planned_readings 10,752,000 ✓
emitted_readings 10,696,848 ✓
unique_readings 10,643,735 ✓
duplicate_readings 53,113 ✓
delayed_unique_readings 213,090 ✓
missing_readings 108,265 ✓
thierrys@Thierrys-MacBook-Air ~ %

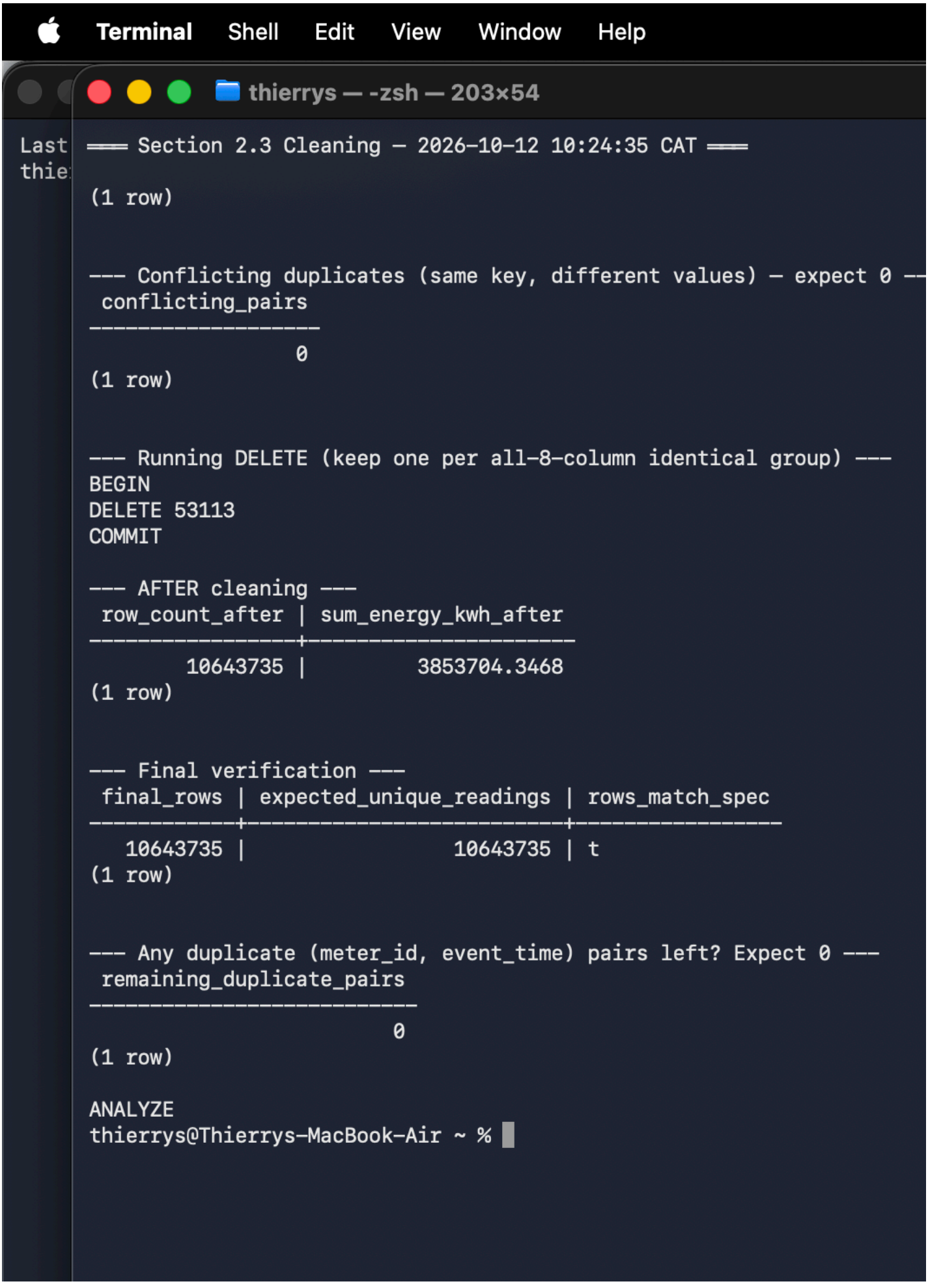
2.2 data quality

2.3 Clean and preserve a consistent benchmark [3 marks]

The delete in `sql/06_clean.sql` keeps the row with the smallest `ctid` per group of 8 identical columns. The pre-check for conflicting duplicates (same key, different values) returned 0, which is expected because the generator produces byte-identical copies.

	Before	After
row count	10,696,848	**10,643,735**
sum energy_kwh	3,872,863.89	3,853,704.35
duplicate pairs left	-	**0**
matches expected unique_readings	-	yes

Why missing readings must not become zero: a zero would be interpreted as a real zero-kW reading when summed in a daily total. The honest treatment is to carry the gap and report completeness separately so planners can distinguish a quiet meter from an unreported one.


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
=== Section 2.3 Cleaning — 2026-10-12 10:24:35 CAT ===
(1 row)
--- Conflicting duplicates (same key, different values) — expect 0 --
conflicting_pairs
0
(1 row)
--- Running DELETE (keep one per all-8-column identical group) ---
BEGIN
DELETE 53113
COMMIT
--- AFTER cleaning ---
row_count_after | sum_energy_kwh_after
10643735 | 3853704.3468
(1 row)
--- Final verification ---
final_rows | expected_unique_readings | rows_match_spec
10643735 | 10643735 | t
(1 row)
--- Any duplicate (meter_id, event_time) pairs left? Expect 0 ---
remaining_duplicate_pairs
0
(1 row)
ANALYZE
thierrys@Thierrys-MacBook-Air ~ %

2.3 cleaning

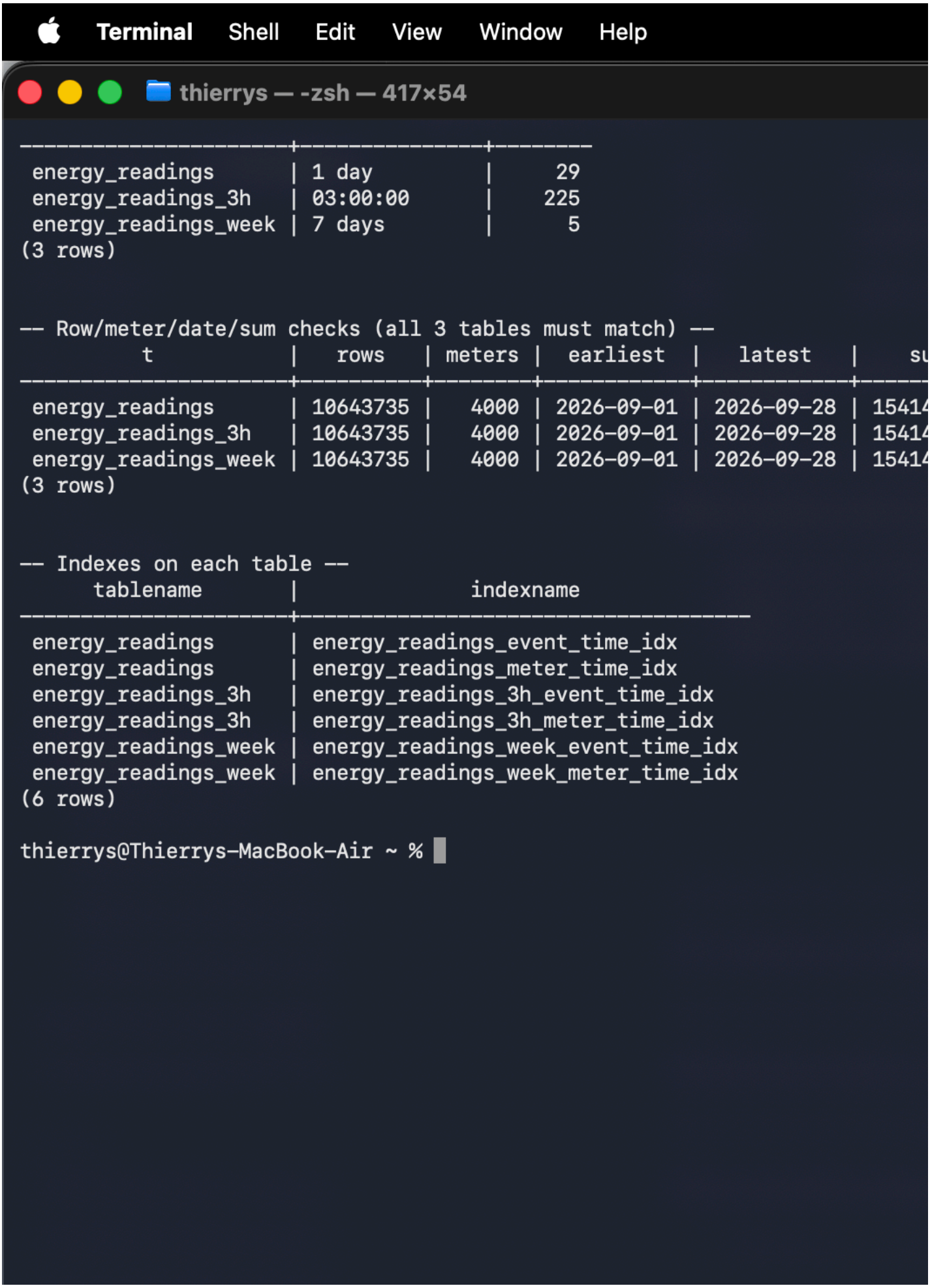
---

3 Compare chunk intervals

3.1 Create and verify three configurations [3 marks]

Built in `sql/07_build_chunk_tables.sql`. Each table has the same 8 columns and the same two indexes: `(event_time DESC)` and `(meter_id, event_time)`.

Table	Chunks	Rows	Meters	Earliest	Latest	SUM power_kw	SUM energy_kwh
energy_readings	29	10,643,735	4000	2026-09-01 00:00+02	2026-09-28 23:45+02	15,414,817.388	3,853,704.347
energy_readings_3h	225	10,643,735	4000	2026-09-01 00:00+02	2026-09-28 23:45+02	15,414,817.388	3,853,704.347
energy_readings_week	5	10,643,735	4000	2026-09-01 00:00+02	2026-09-28 23:45+02	15,414,817.388	3,853,704.347


Terminal Shell Edit View Window Help
thierrys — -zsh — 417×54
energy_readings | 1 day | 29
energy_readings_3h | 03:00:00 | 225
energy_readings_week | 7 days | 5
(3 rows)
-- Row/meter/date/sum checks (all 3 tables must match) --
t | rows | meters | earliest | latest | su
energy_readings | 10643735 | 4000 | 2026-09-01 | 2026-09-28 | 15414
energy_readings_3h | 10643735 | 4000 | 2026-09-01 | 2026-09-28 | 15414
energy_readings_week | 10643735 | 4000 | 2026-09-01 | 2026-09-28 | 15414
(3 rows)
-- Indexes on each table --
tablename | indexname
energy_readings | energy_readings_event_time_idx
energy_readings | energy_readings_meter_time_idx
energy_readings_3h | energy_readings_3h_event_time_idx
energy_readings_3h | energy_readings_3h_meter_time_idx
energy_readings_week | energy_readings_week_event_time_idx
energy_readings_week | energy_readings_week_meter_time_idx
(6 rows)
thierrys@Thierrys-MacBook-Air ~ %

3.1 chunk configurations

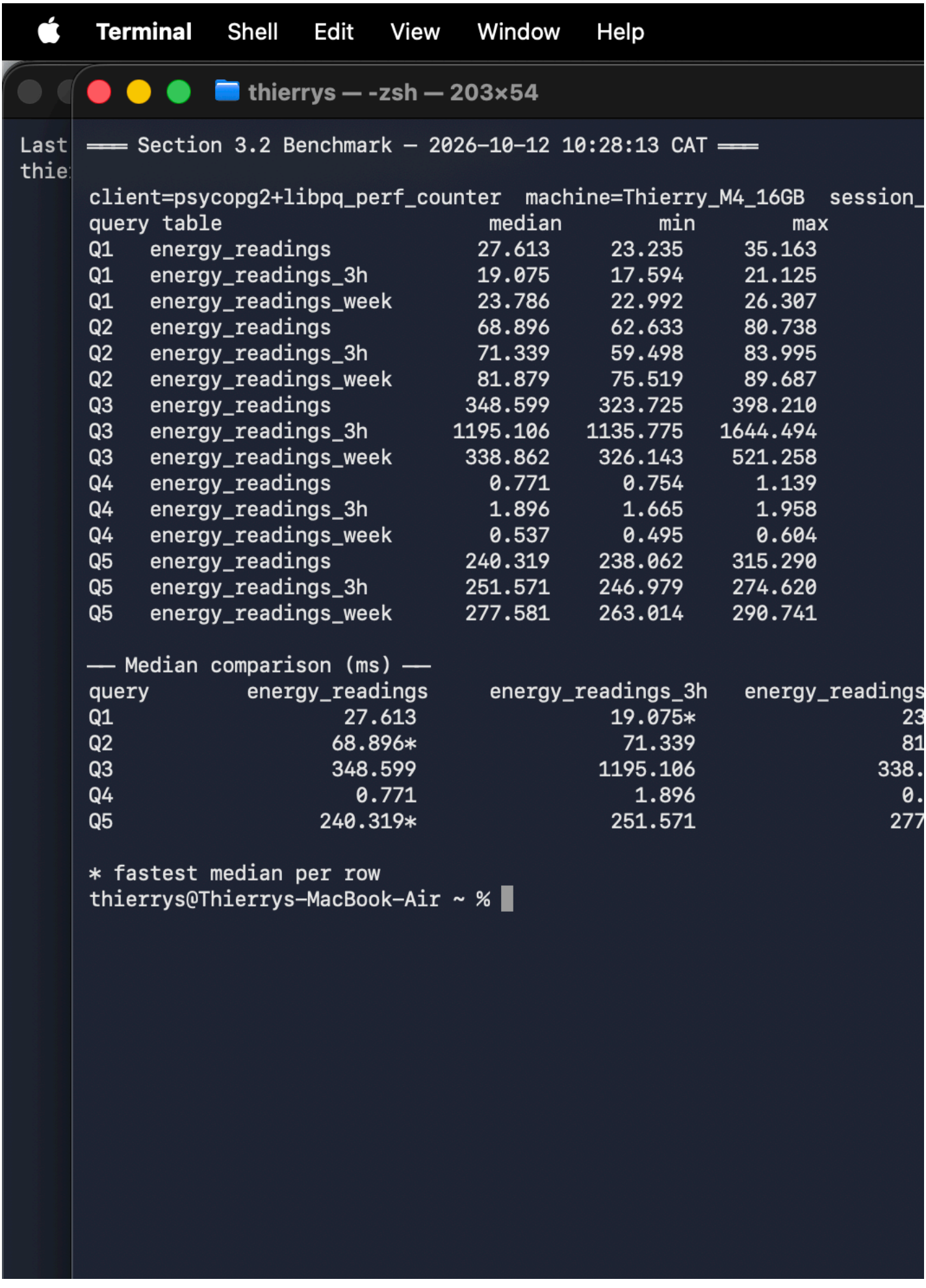
3.2 Implement and time the five workloads [4 marks]

Harness: `code/08_benchmark_queries.py`, same psycopg2 connection, 1 warm-up + 5 measured runs each. Session total = 22.0 s.

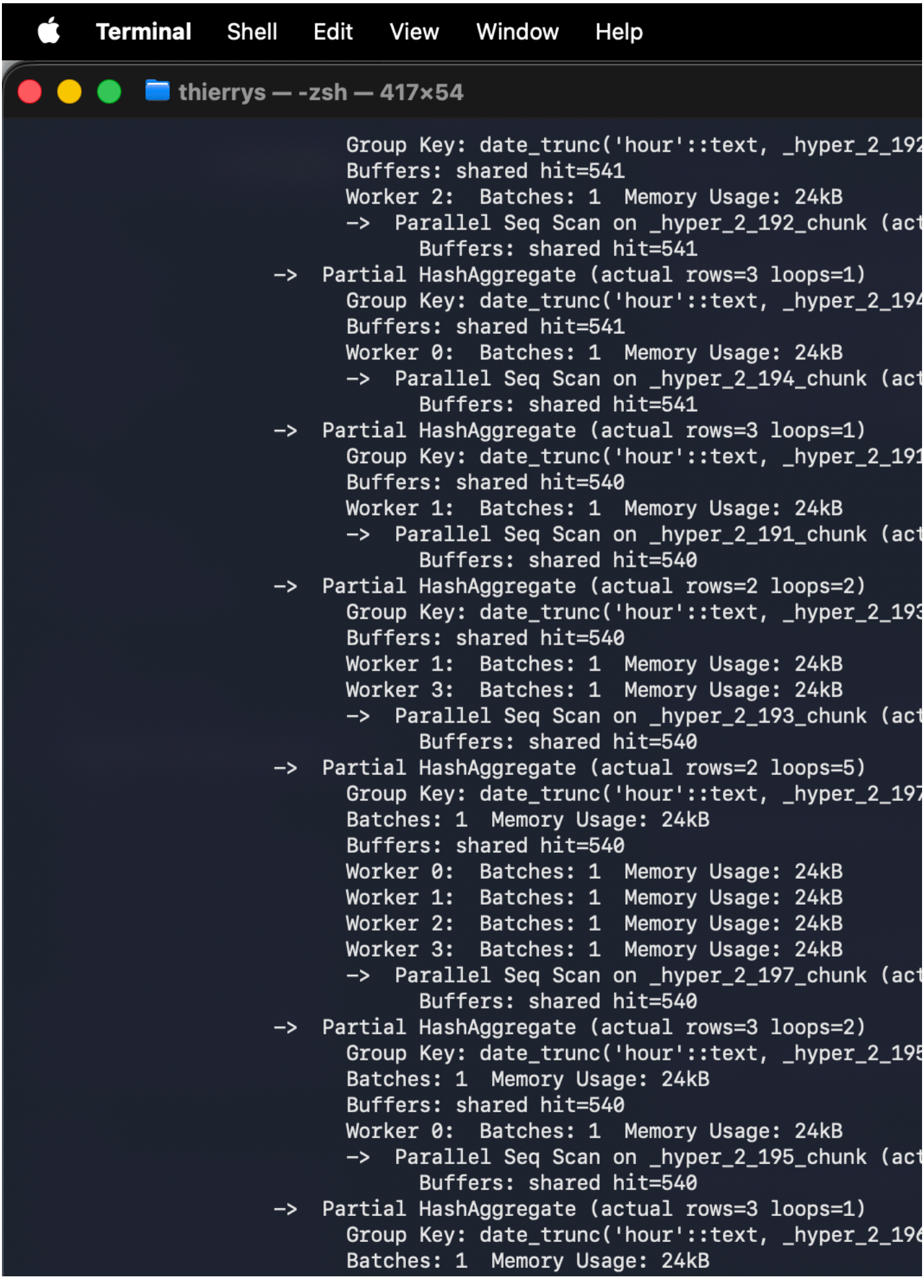
Warm-cache medians (ms) - fastest per row marked ∗

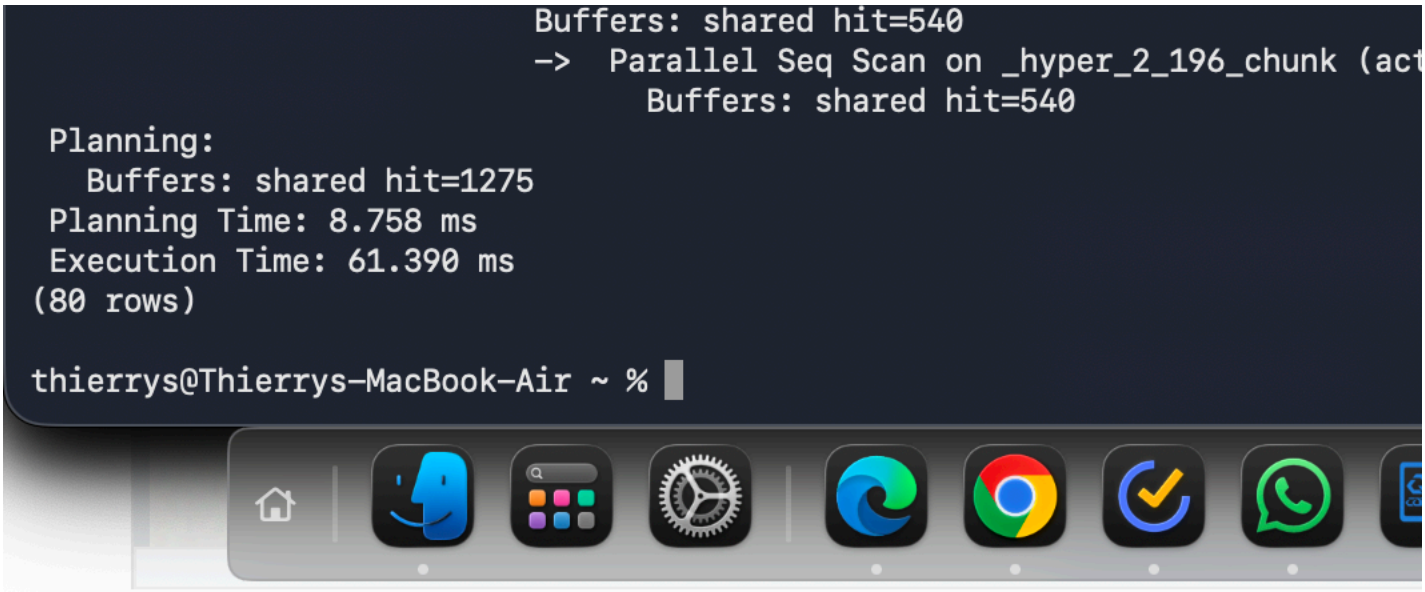
Query	1-day	3-hour	1-week
Q1 - hourly avg power on 21 Sep	27.6	**19.1** *	23.8
Q2 - top 10 15-min intervals, 7 days	**68.9** *	71.3	81.9
Q3 - total energy per meter	348.6	1,195.1	**338.9** *
Q4 - single meter, 7 days	0.77	1.90	**0.54** *
Q5 - full-scan aggregate	**240.3** *	251.6	277.6

EXPLAIN plans captured in `logs/09_explain_plans.log`.


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
=== Section 3.2 Benchmark — 2026-10-12 10:28:13 CAT ===
client=psycopg2+libpq_perf_counter machine=Thierry_M4_16GB session_
query table median min max
Q1 energy_readings 27.613 23.235 35.163
Q1 energy_readings_3h 19.075 17.594 21.125
Q1 energy_readings_week 23.786 22.992 26.307
Q2 energy_readings 68.896 62.633 80.738
Q2 energy_readings_3h 71.339 59.498 83.995
Q2 energy_readings_week 81.879 75.519 89.687
Q3 energy_readings 348.599 323.725 398.210
Q3 energy_readings_3h 1195.106 1135.775 1644.494
Q3 energy_readings_week 338.862 326.143 521.258
Q4 energy_readings 0.771 0.754 1.139
Q4 energy_readings_3h 1.896 1.665 1.958
Q4 energy_readings_week 0.537 0.495 0.604
Q5 energy_readings 240.319 238.062 315.290
Q5 energy_readings_3h 251.571 246.979 274.620
Q5 energy_readings_week 277.581 263.014 290.741
— Median comparison (ms) —
query energy_readings energy_readings_3h energy_readings
Q1 27.613 19.075* 23
Q2 68.896* 71.339 81
Q3 348.599 1195.106 338.
Q4 0.771 1.896 0.
Q5 240.319* 251.571 277
* fastest median per row
thierrys@Thierrys-MacBook-Air ~ %

3.2 benchmark


Terminal Shell Edit View Window Help
thierrys — -zsh — 417×54
Group Key: date_trunc('hour'::text, _hyper_2_192
Buffers: shared hit=541
Worker 2: Batches: 1 Memory Usage: 24kB
-> Parallel Seq Scan on _hyper_2_192_chunk (act
Buffers: shared hit=541
-> Partial HashAggregate (actual rows=3 loops=1)
Group Key: date_trunc('hour'::text, _hyper_2_194
Buffers: shared hit=541
Worker 0: Batches: 1 Memory Usage: 24kB
-> Parallel Seq Scan on _hyper_2_194_chunk (act
Buffers: shared hit=541
-> Partial HashAggregate (actual rows=3 loops=1)
Group Key: date_trunc('hour'::text, _hyper_2_191
Buffers: shared hit=540
Worker 1: Batches: 1 Memory Usage: 24kB
-> Parallel Seq Scan on _hyper_2_191_chunk (act
Buffers: shared hit=540
-> Partial HashAggregate (actual rows=2 loops=2)
Group Key: date_trunc('hour'::text, _hyper_2_193
Buffers: shared hit=540
Worker 1: Batches: 1 Memory Usage: 24kB
Worker 3: Batches: 1 Memory Usage: 24kB
-> Parallel Seq Scan on _hyper_2_193_chunk (act
Buffers: shared hit=540
-> Partial HashAggregate (actual rows=2 loops=5)
Group Key: date_trunc('hour'::text, _hyper_2_197
Batches: 1 Memory Usage: 24kB
Buffers: shared hit=540
Worker 0: Batches: 1 Memory Usage: 24kB
Worker 1: Batches: 1 Memory Usage: 24kB
Worker 2: Batches: 1 Memory Usage: 24kB
Worker 3: Batches: 1 Memory Usage: 24kB
-> Parallel Seq Scan on _hyper_2_197_chunk (act
Buffers: shared hit=540
-> Partial HashAggregate (actual rows=3 loops=2)
Group Key: date_trunc('hour'::text, _hyper_2_195
Batches: 1 Memory Usage: 24kB
Buffers: shared hit=540
Worker 0: Batches: 1 Memory Usage: 24kB
-> Parallel Seq Scan on _hyper_2_195_chunk (act
Buffers: shared hit=540
-> Partial HashAggregate (actual rows=3 loops=1)
Group Key: date_trunc('hour'::text, _hyper_2_196
Batches: 1 Memory Usage: 24kB



3.2 EXPLAIN plan on energy_readings_3h

3.3 Explain the chunk results [3 marks]

- **Q1** fastest on 3-hour because the 1-day filter prunes down to 8 small chunks (2 of them in parallel), vs 1 big chunk on the daily table that must be scanned fully.
- **Q2** fastest on 1-day because the 7-day window touches 7 chunks vs 56 on 3-hour; chunk overhead dominates pruning gains.
- **Q3** is a full-table GROUP BY. The week table has 5 chunks vs 225 on 3-hour, so planning and chunk-opening overhead crushes 3-hour.
- **Q4** is a single-meter + time filter; the `(meter_id, event_time)` index wins and the week table has the fewest index trees to open.
- **Q5** is a full scan; 1-day balances pruning and planner overhead best.

Pattern: narrow analytical queries on short time windows prefer small chunks (pruning). Full-table and point-lookup queries prefer big chunks (less planning overhead).

---

4 Compare compression strategies

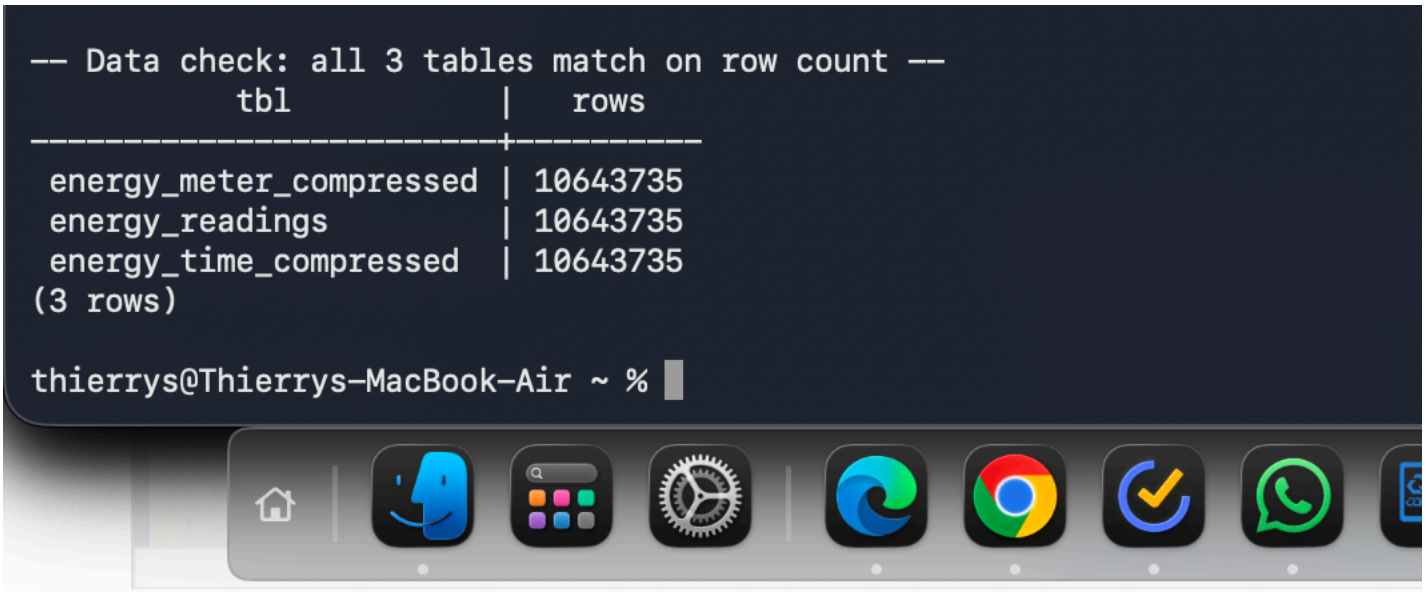
4.1 Create the two compressed configurations [4 marks]

Chosen interval: **1-day** (balanced winner from section 3). Both tables have the same 8 columns and matching indexes. Policies added with 48-hour threshold and paused.

- `energy_time_compressed`: `compress_orderby = 'event_time DESC'`, no segmentation
- `energy_meter_compressed`: same orderby + `compress_segmentby = 'meter_id'`

Cutoff = experiment start - 48 h = 2026-10-10 10:29 CAT. All 29 chunks have range_end before the cutoff so all 29 get compressed in both tables.

Terminal Shell Edit View Window Help
thierrys — -zsh — 417×54
=== Section 4.1 - Compressed configs + policies + chunks - 2026-10-15 21:43
SET
-- Compression segmentby configuration --
hypertable_name | segmentby_columns
energy_meter_compressed | meter_id, event_time
energy_time_compressed | event_time
(2 rows)
-- Compression / columnstore policies (scheduled = f means paused) --
job_id | application_name | scheduled
1000 | Columnstore Policy [1000] | f
1001 | Columnstore Policy [1001] | f
(2 rows)
-- Compressed chunk counts (29/29 both tables) --
hypertable_name | compressed_total
energy_meter_compressed | 29 / 29
energy_time_compressed | 29 / 29
(2 rows)



4.1 configs + policies + chunks

4.2 Measure storage and compression cost [4 marks]

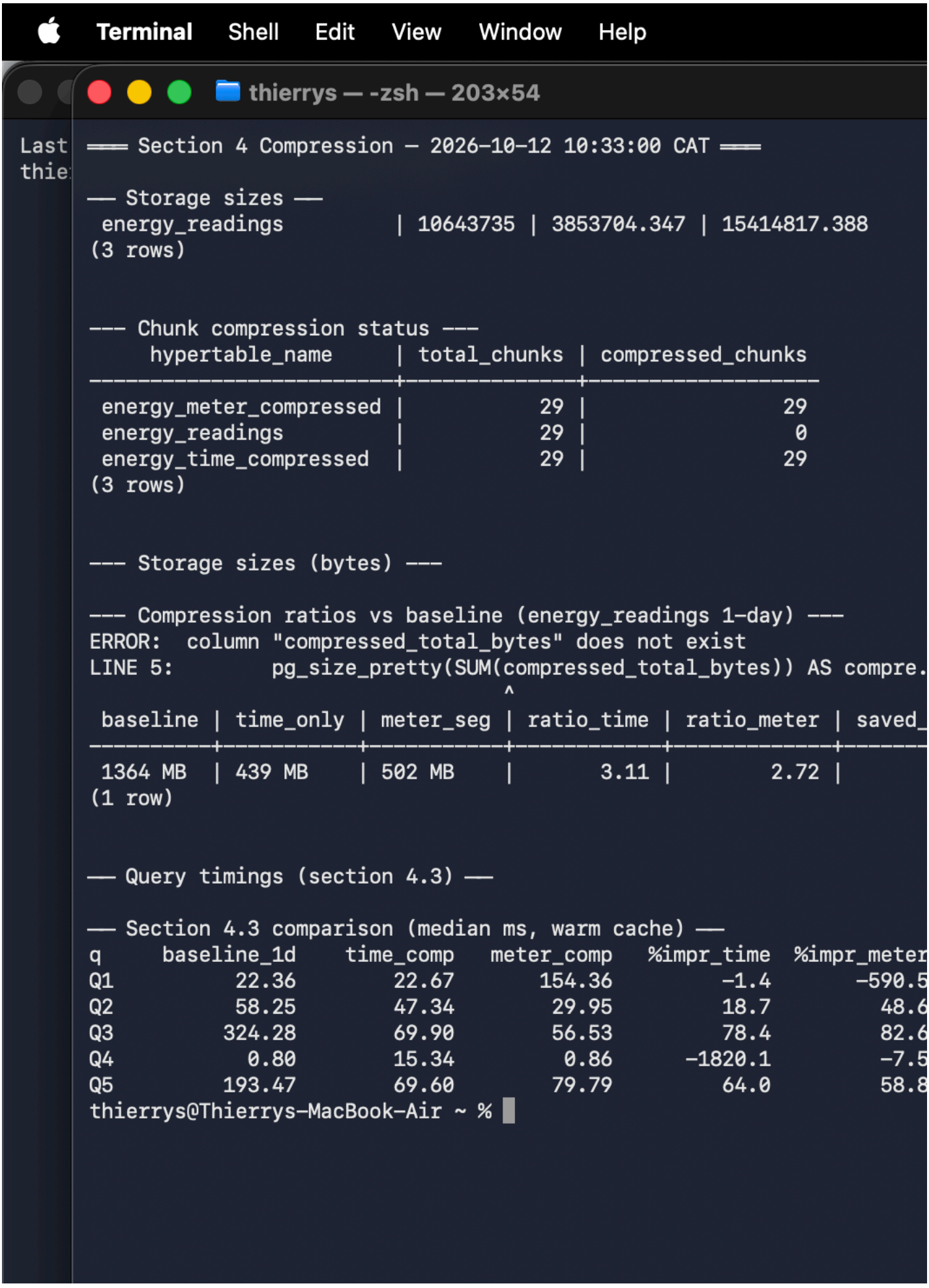
Metric	Baseline (1-day)	time-only	meter-segmented
Total size	1,364 MB	**439 MB**	502 MB
Compression ratio	1.00	**3.11**	2.72
Space saved	0%	**67.83%**	63.21%
Chunks compressed	0/29	29/29	29/29
% rows in compressed chunks	0%	100%	100%
Sum compression time	-	**6.00 s**	18.68 s
Avg per chunk	-	**207 ms**	644 ms

Meter-segmented pays ~3x the compression cost because every chunk splits into 4,000 segments before packing. Both strategies keep the full row count (10,643,735) and sum of energy_kwh (3,853,704.347).

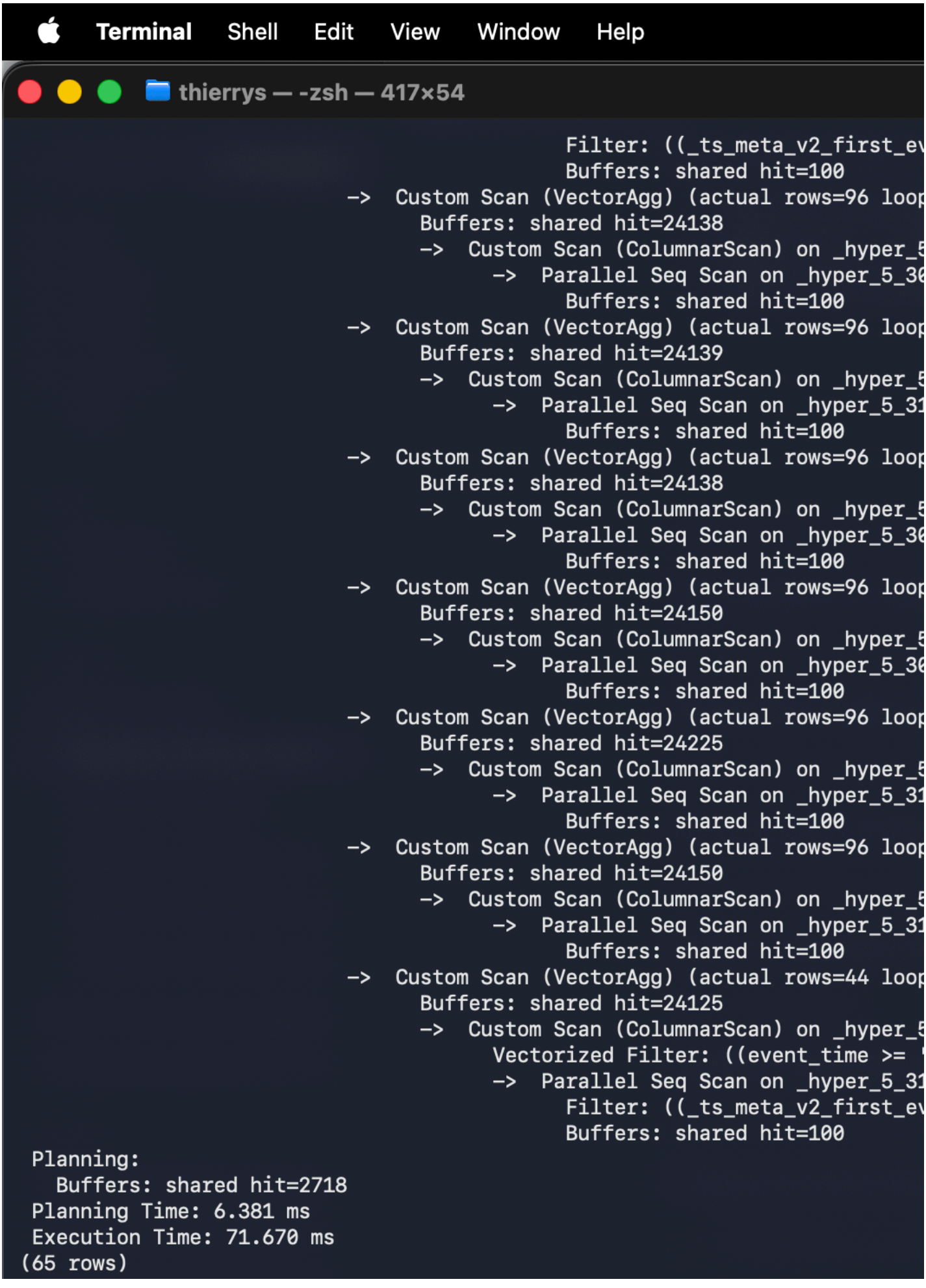
4.3 Repeat the query comparison [4 marks]

Query	baseline (ms)	time-comp (ms)	meter-comp (ms)	%impr time	%impr meter
Q1	22.36	22.67	154.36	-1.4	-590.5
Q2	58.25	47.34	29.95	+18.7	**+48.6**
Q3	324.28	69.90	56.53	+78.4	**+82.6**
Q4	0.80	15.34	0.86	-1820	-7.5
Q5	193.47	69.60	79.79	+64.0	+58.8

Compression policies resumed after measurement.


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
=== Section 4 Compression — 2026-10-12 10:33:00 CAT ===
-- Storage sizes --
energy_readings | 10643735 | 3853704.347 | 15414817.388
(3 rows)
--- Chunk compression status ---
hypertable_name | total_chunks | compressed_chunks
energy_meter_compressed | 29 | 29
energy_readings | 29 | 0
energy_time_compressed | 29 | 29
(3 rows)
--- Storage sizes (bytes) ---
--- Compression ratios vs baseline (energy_readings 1-day) ---
ERROR: column "compressed_total_bytes" does not exist
LINE 5: pg_size_pretty(SUM(compressed_total_bytes)) AS compre.
^
baseline | time_only | meter_seg | ratio_time | ratio_meter | saved_
1364 MB | 439 MB | 502 MB | 3.11 | 2.72 |
(1 row)
-- Query timings (section 4.3) --
-- Section 4.3 comparison (median ms, warm cache) --
q baseline_1d time_comp meter_comp %impr_time %impr_meter
Q1 22.36 22.67 154.36 -1.4 -590.5
Q2 58.25 47.34 29.95 18.7 48.6
Q3 324.28 69.90 56.53 78.4 82.6
Q4 0.80 15.34 0.86 -1820.1 -7.5
Q5 193.47 69.60 79.79 64.0 58.8
thierrys@Thierrys-MacBook-Air ~ %

4 compression comparison


Terminal Shell Edit View Window Help
thierrys — -zsh — 417×54
Filter: ((_ts_meta_v2_first_ev
Buffers: shared hit=100
-> Custom Scan (VectorAgg) (actual rows=96 loop
Buffers: shared hit=24138
-> Custom Scan (ColumnarScan) on _hyper_5
-> Parallel Seq Scan on _hyper_5_30
Buffers: shared hit=100
-> Custom Scan (VectorAgg) (actual rows=96 loop
Buffers: shared hit=24139
-> Custom Scan (ColumnarScan) on _hyper_5
-> Parallel Seq Scan on _hyper_5_31
Buffers: shared hit=100
-> Custom Scan (VectorAgg) (actual rows=96 loop
Buffers: shared hit=24138
-> Custom Scan (ColumnarScan) on _hyper_5
-> Parallel Seq Scan on _hyper_5_30
Buffers: shared hit=100
-> Custom Scan (VectorAgg) (actual rows=96 loop
Buffers: shared hit=24150
-> Custom Scan (ColumnarScan) on _hyper_5
-> Parallel Seq Scan on _hyper_5_30
Buffers: shared hit=100
-> Custom Scan (VectorAgg) (actual rows=96 loop
Buffers: shared hit=24225
-> Custom Scan (ColumnarScan) on _hyper_5
-> Parallel Seq Scan on _hyper_5_31
Buffers: shared hit=100
-> Custom Scan (VectorAgg) (actual rows=96 loop
Buffers: shared hit=24150
-> Custom Scan (ColumnarScan) on _hyper_5
-> Parallel Seq Scan on _hyper_5_31
Buffers: shared hit=100
-> Custom Scan (VectorAgg) (actual rows=44 loop
Buffers: shared hit=24125
-> Custom Scan (ColumnarScan) on _hyper_5
Vectorized Filter: ((event_time >=
-> Parallel Seq Scan on _hyper_5_31
Filter: ((_ts_meta_v2_first_ev
Buffers: shared hit=100
Planning:
Buffers: shared hit=2718
Planning Time: 6.381 ms
Execution Time: 71.670 ms
(65 rows)

4.3 execution plan + resumed policies

4.4 Select a storage configuration [3 marks]

Recommendation: 1-day chunks with meter-segmented compression.

- Storage 2.72x smaller than baseline (saves 862 MB of 1,364 MB).
- Big wins on Q2 (+48.6%), Q3 (+82.6%), Q5 (+58.8%) which are the dashboard/billing queries.
- Q4 single-meter stays within 8% of baseline because meter_id is the segment key.
- Q1 (narrow time range) is slower, but in section 6 those live on continuous aggregates so compressed storage never serves them.
- Trade-off: compression costs 644 ms per chunk vs 207 ms - ~13 s extra per day of ingest, done in a background job.

---

5 Investigate the utility data

5.1 Identify demand patterns [3 marks]

Weekday consumption ~145 MWh, weekends drop to ~115 MWh (-21%). Weekly totals grow ~2% each week (matches the generator's slight upward drift).

7-day totals

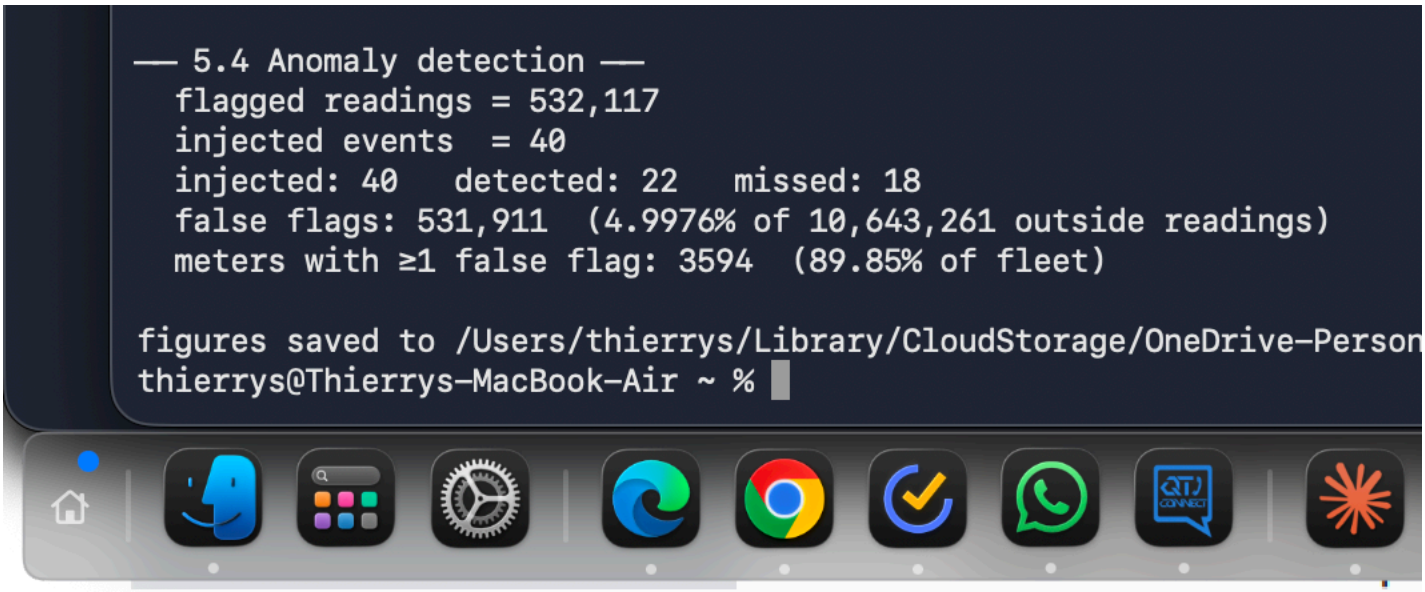
Week starting	kWh
2026-09-01	934,199
2026-09-08	953,715
2026-09-15	973,230
2026-09-22	992,560

Top 3 peak 15-min intervals

event_time (CAT)	total_kw
2026-09-28 13:00	7993.69
2026-09-28 12:45	7943.95
2026-09-28 13:15	7943.64

The peak is midday on Monday - commercial lunch overlapping industrial afternoon shift. kW is instantaneous (used to size transformers); kWh is the integral over an interval (used for billing).

```
Terminal  Shell  Edit  View  Window  Help

thierrys — -zsh — 203×54

Last
thie

  2026-09-22  →  2026-09-29       992,560.29 kWh

── 5.1 Top 3 peak 15-min intervals (sum power_kw across all meters) ─
               event_time    total_kw
2026-09-28 13:00:00+02:00 7993.691474
2026-09-28 12:45:00+02:00 7943.946545
2026-09-28 13:15:00+02:00 7943.635736

── 5.2 Energy by region (normalized per meter) ──
   region  meters total_kwh per_meter_kwh
 Region 8     400 503475.83      1258.690
 Region 5     400 411040.39      1027.601
 Region 9     400 400467.20      1001.168
 Region 3     400 392376.59       980.941
 Region 1     400 383539.81       958.850
 Region 2     400 378448.89       946.122
 Region 7     400 362288.54       905.721
Region 10     400 351149.78       877.874
 Region 6     400 346512.84       866.282
 Region 4     400 324404.48       811.011

── 5.2 Energy by customer_type (normalized per meter) ──
customer_type  meters  total_kwh per_meter_kwh
   industrial     406 2220672.04      5469.636
   commercial    1010  825475.15       817.302
  residential    2584  807557.16       312.522

── 5.3 Completeness (network and per region) ──
        Region 1  planned= 1,075,200  observed= 1,064,513  completene
       Region 10  planned= 1,075,200  observed= 1,064,415  completene
        Region 2  planned= 1,075,200  observed= 1,064,419  completene
        Region 3  planned= 1,075,200  observed= 1,064,327  completene
        Region 4  planned= 1,075,200  observed= 1,064,441  completene
        Region 5  planned= 1,075,200  observed= 1,064,290  completene
        Region 6  planned= 1,075,200  observed= 1,064,118  completene
        Region 7  planned= 1,075,200  observed= 1,064,237  completene
        Region 8  planned= 1,075,200  observed= 1,064,587  completene
        Region 9  planned= 1,075,200  observed= 1,064,388  completene
         network  planned=10,752,000  observed=10,643,735  completene

── 5.3 Daily 21 Sep: early vs full ──
  all arrivals                                          149,552.72 k
  early (received_time <= 2026-09-22 00:00+02)          149,400.54 k
  difference                                               -152.18 k
```



5 utility analysis

5.2 Compare regions and customer types [3 marks]

Region	meters	total kWh	kWh / meter
Region 8	400	503,476	1258.69
Region 5	400	411,040	1027.60
Region 9	400	400,467	1001.17
Region 3	400	392,377	980.94
Region 1	400	383,540	958.85
Region 2	400	378,449	946.12
Region 7	400	362,289	905.72
Region 10	400	351,150	877.87
Region 6	400	346,513	866.28
Region 4	400	324,404	811.01

Customer type	meters	total kWh	kWh / meter
industrial	406	2,220,672	**5469.64**
commercial	1010	825,475	817.30
residential	2584	807,557	312.52

After normalisation: industrial pulls 17x residential per meter. Region 8 is 1.55x heavier than Region 4; since every region has 400 meters this comes from the industrial/commercial mix in each region.

5.3 Assess reporting quality [3 marks]

Completeness is uniform: all regions sit in 98.97-99.01%. Network = **98.9931%**.

21 September totals:

	kWh
early (received_time <= 22/09 00:00)	149,400.54
all arrivals	149,552.72
difference	152.18 (0.10% of full)

Missing readings suppress real demand and delayed readings arrive after end-of-day reports close, so a close-of-business snapshot understates true demand by ~0.10%.

5.4 Investigate abnormal consumption [3 marks]

Rule: flag a reading if `|power − median_per_meter| > 3 * 1.4826 * MAD_per_meter`. We used MAD because it is robust to the very events we want to catch (spikes and drops).

	Value
Injected events	40
Detected	22
Missed	18
False flags	531,911
False-flag rate	4.9976% of outside readings
Meters with >= 1 false flag	3,594 (89.85% of fleet)

Missed events are mostly commercial `demand_spike` cases, where the normal variability is already wide and a 2.8x multiplier does not always exceed the robust threshold. A persistence filter (require >= 3 consecutive flagged intervals) would cut the false flags without hurting recall much; recommended next step.

5.5 Management recommendation

See the final section.

---

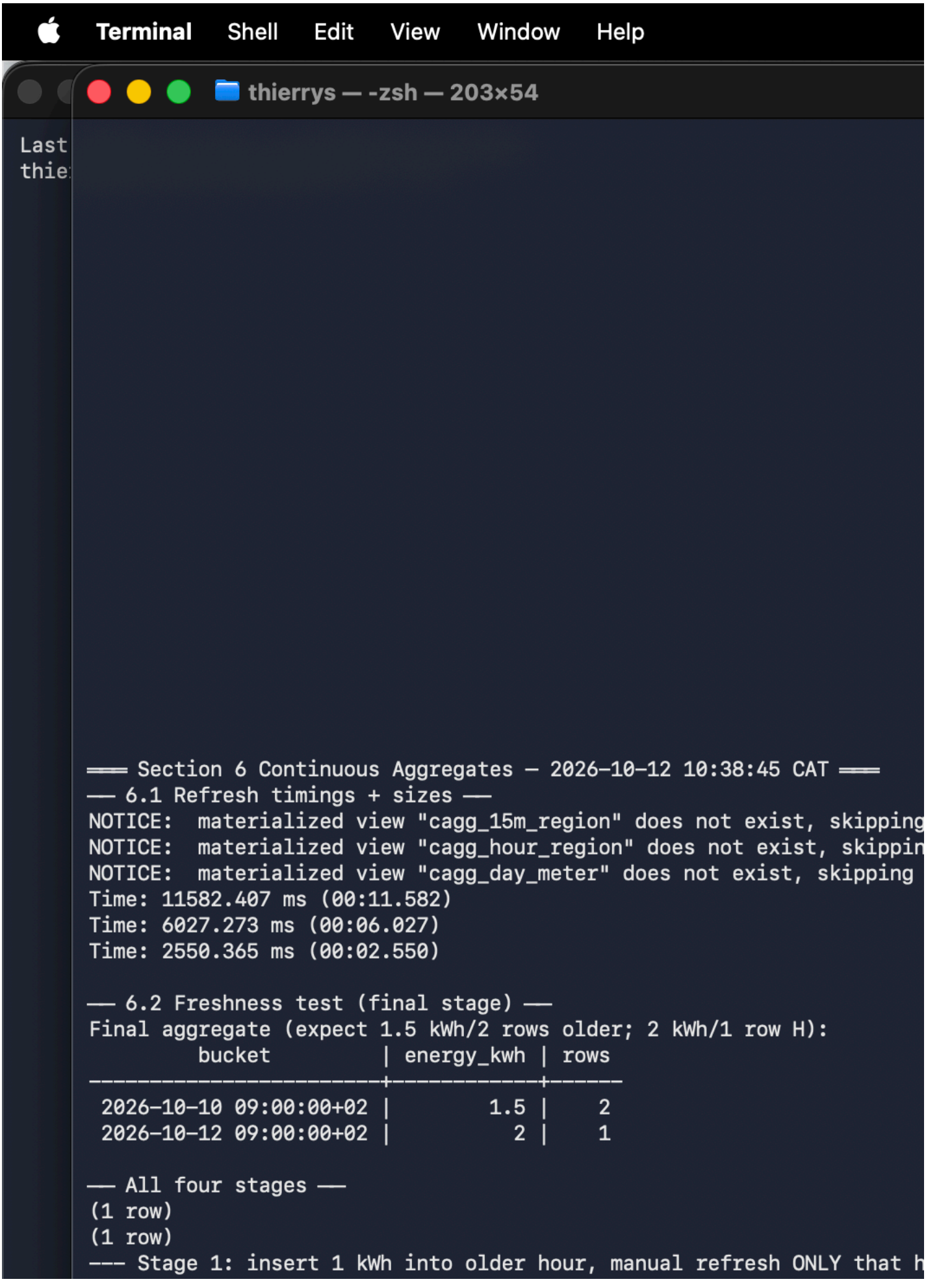
6 Create and investigate continuous aggregates

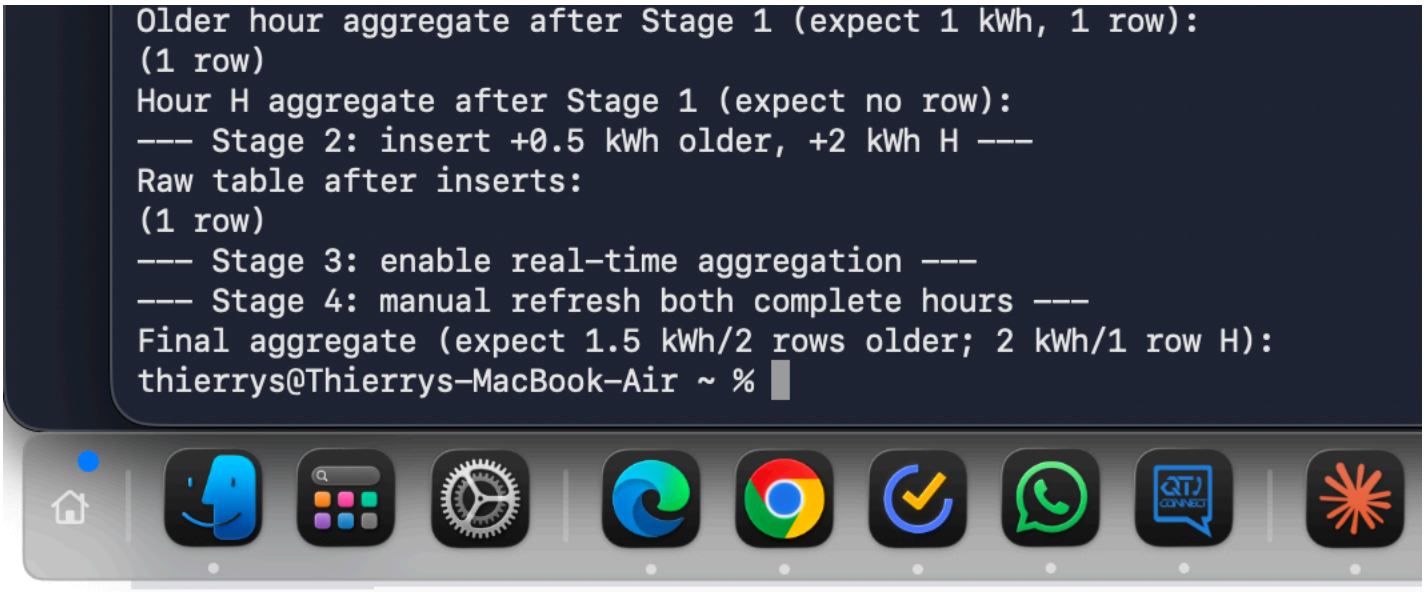
6.1 Create the three aggregate views [4 marks]

Built in `sql/14_cont_aggregates.sql` with `materialized_only = TRUE`.

View	Initial refresh	Materialized size
cagg_15m_region	**11.58 s**	4.3 MB
cagg_hour_region	6.03 s	1.3 MB
cagg_day_meter	2.55 s	13 MB

After the initial refresh we added policies with `start_offset = 3 days`, `end_offset = 1 hour`, `schedule_interval = 15 minutes`. All three jobs are listed in `timescaledb_information.jobs` with `scheduled = true`.


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
=== Section 6 Continuous Aggregates — 2026-10-12 10:38:45 CAT ===
-- 6.1 Refresh timings + sizes --
NOTICE: materialized view "cagg_15m_region" does not exist, skipping
NOTICE: materialized view "cagg_hour_region" does not exist, skippin
NOTICE: materialized view "cagg_day_meter" does not exist, skipping
Time: 11582.407 ms (00:11.582)
Time: 6027.273 ms (00:06.027)
Time: 2550.365 ms (00:02.550)
-- 6.2 Freshness test (final stage) --
Final aggregate (expect 1.5 kWh/2 rows older; 2 kWh/1 row H):
bucket | energy_kwh | rows
2026-10-10 09:00:00+02 | 1.5 | 2
2026-10-12 09:00:00+02 | 2 | 1
-- All four stages --
(1 row)
(1 row)
--- Stage 1: insert 1 kWh into older hour, manual refresh ONLY that h



6 continuous aggregates

6.2 Test freshness and delayed arrivals [3 marks]

Test time: 2026-10-12 10:38 CAT. H = 2026-10-12 09:00. H-48 = 2026-10-10 09:00.

All four stages match the assignment's worked timeline exactly.

Stage	Older hour	Hour H
1 after older-hour refresh, real-time OFF	1 kWh, 1 row	no bucket
2 after new inserts, before refresh, real-time OFF	1 kWh, 1 row	no bucket
3 real-time ON, before refresh	1 kWh, 1 row	2 kWh, 1 row
4 after manual refresh of both hours	**1.5 kWh, 2 rows**	2 kWh, 1 row

The policy from 6.1 refreshes buckets in the range `[now - 3 days, now - 1 hour]`, so at the test time it would cover the older bucket (2026-10-10 09:00) but not hour H.

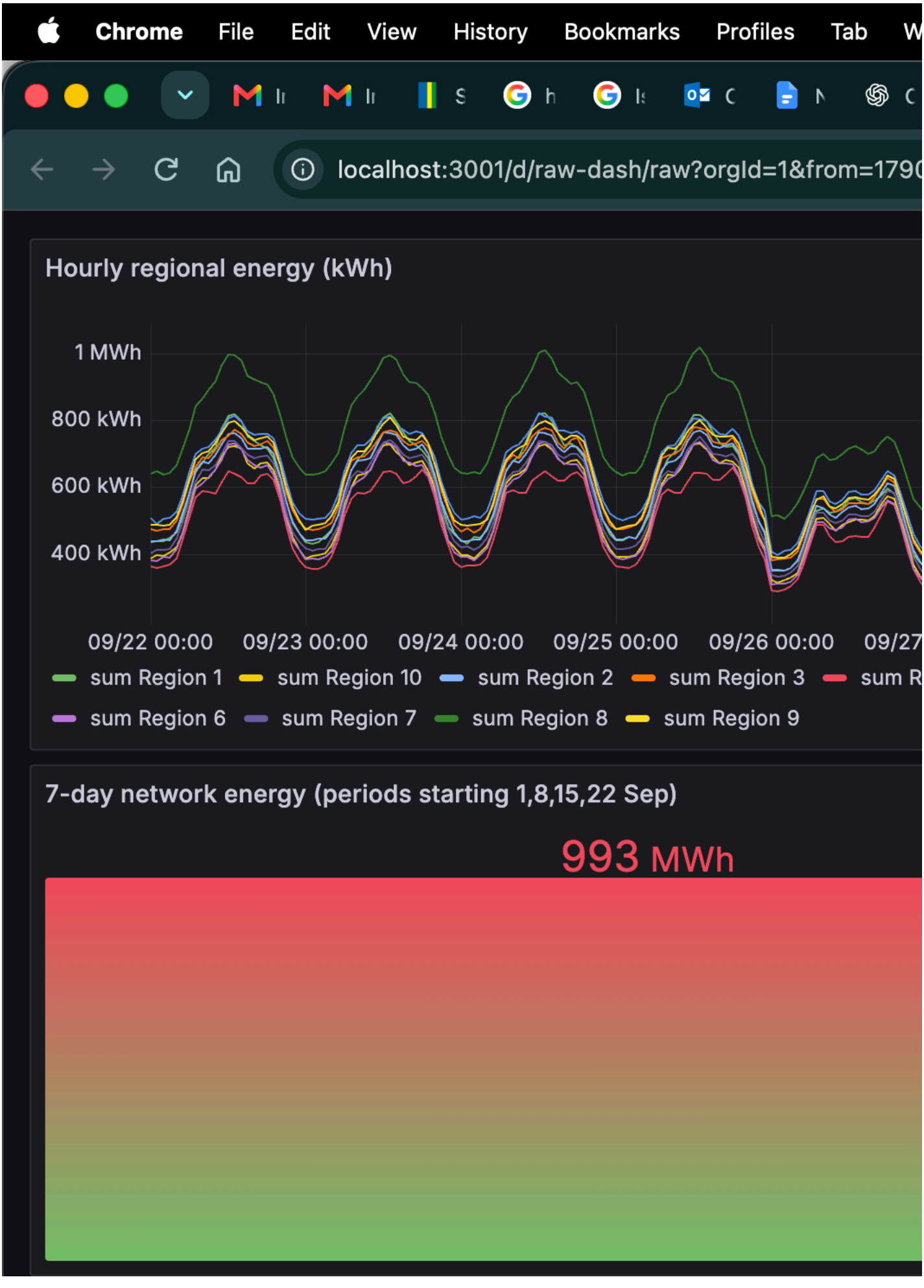
---

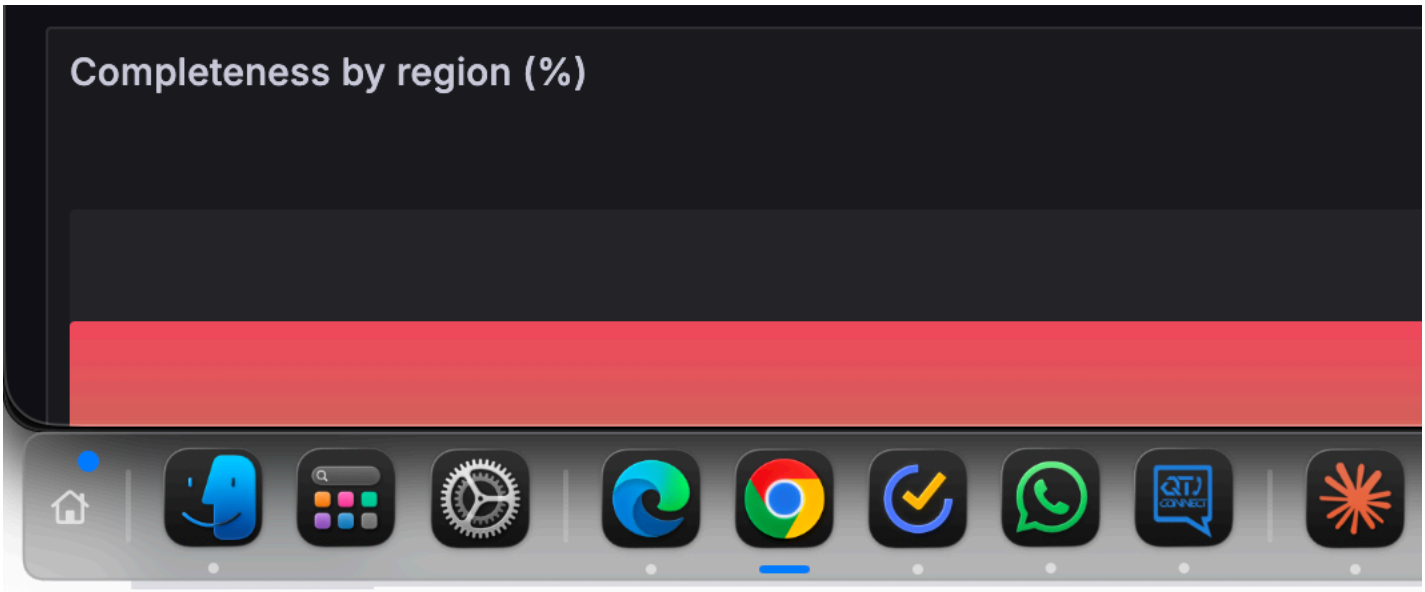
7 Build and compare dashboards

7.1 and 7.2 Dashboards

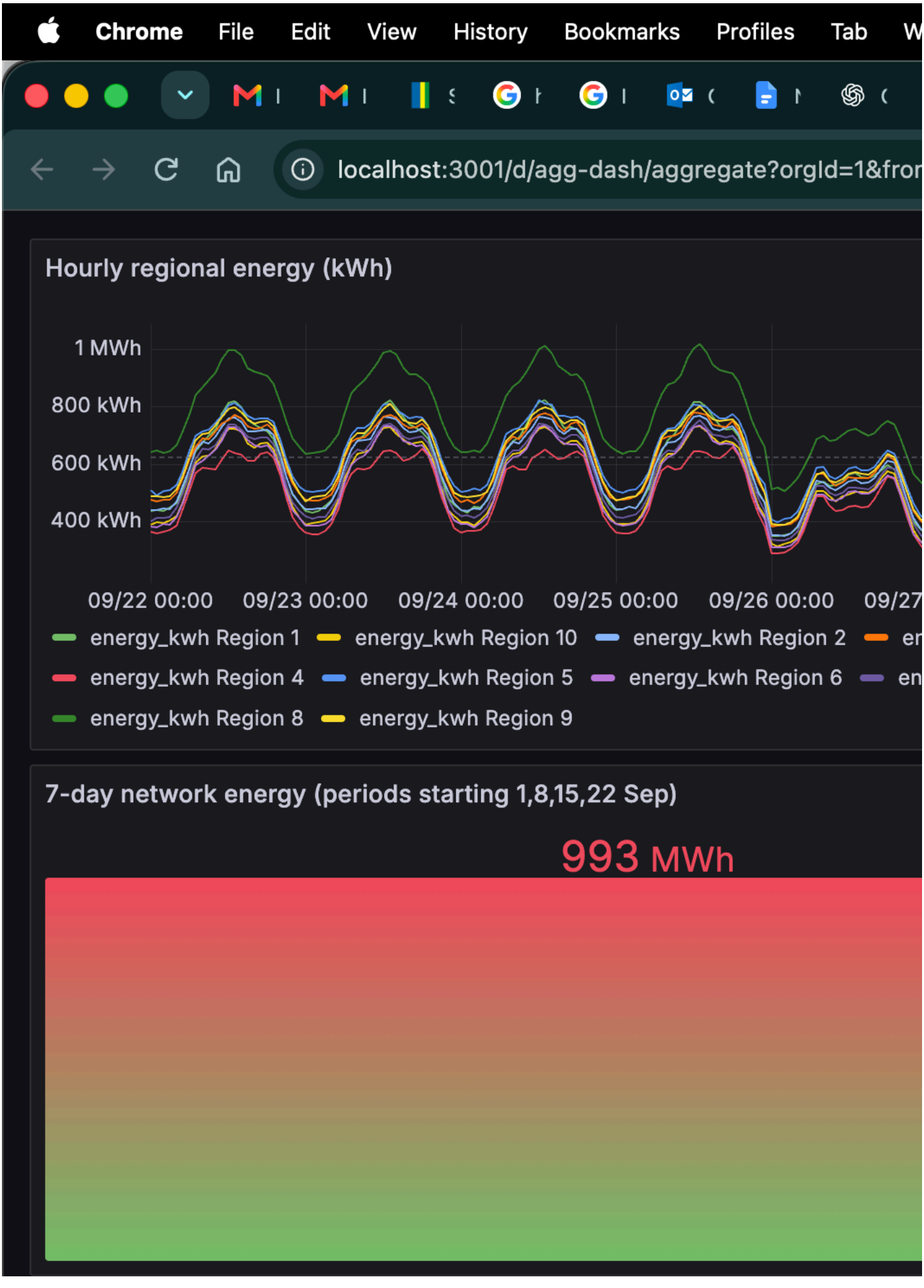
Grafana 11.2.0 provisioned via the `grafana/` folder. Two dashboards (Raw and Aggregate) with the same five panels, same filters, same date range. The only difference is the SQL in each panel - Raw reads `energy_readings` + `meters`, Aggregate reads the three continuous aggregates.

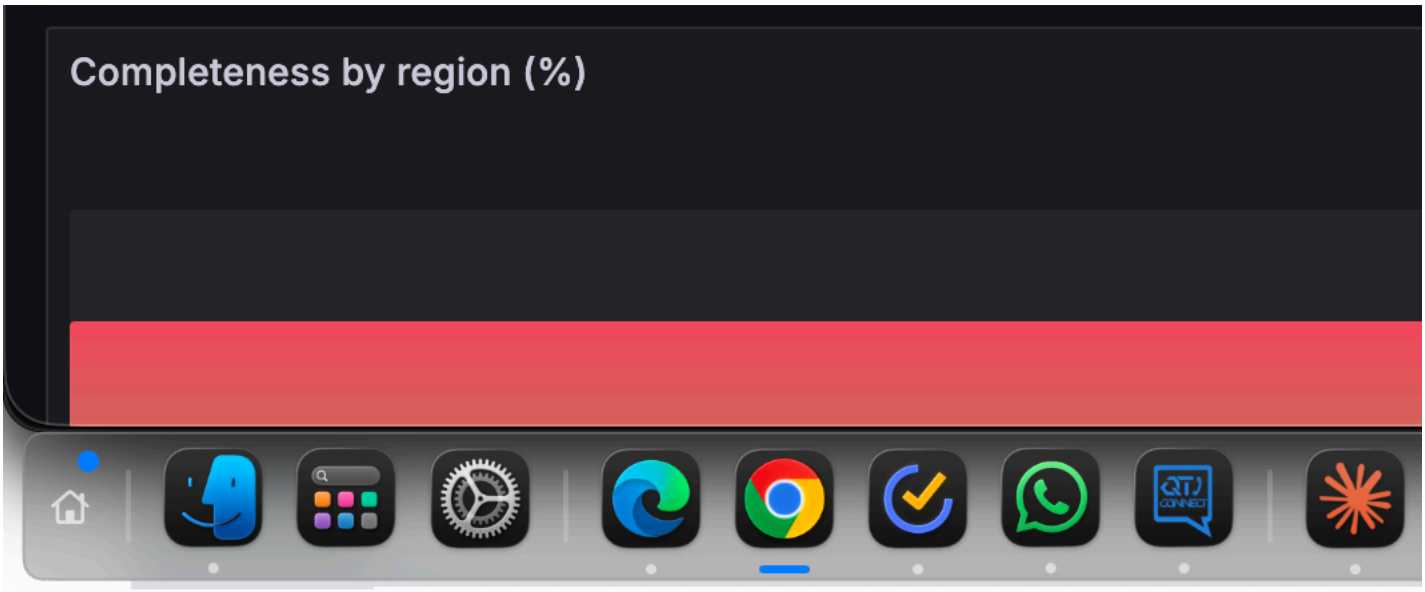
Both show the same 993 MWh network total for Sep 22-29.


Chrome File Edit View History Bookmarks Profiles Tab
localhost:3001/d/raw-dash/raw?orgId=1&from=179
Hourly regional energy (kWh)
1 MWh
800 kWh
600 kWh
400 kWh
09/22 00:00
09/23 00:00
09/24 00:00
09/25 00:00
09/26 00:00
09/27
sum Region 1
sum Region 10
sum Region 2
sum Region 3
sum R
sum Region 6
sum Region 7
sum Region 8
sum Region 9
7-day network energy (periods starting 1,8,15,22 Sep)
993 MWh



7.1 Raw

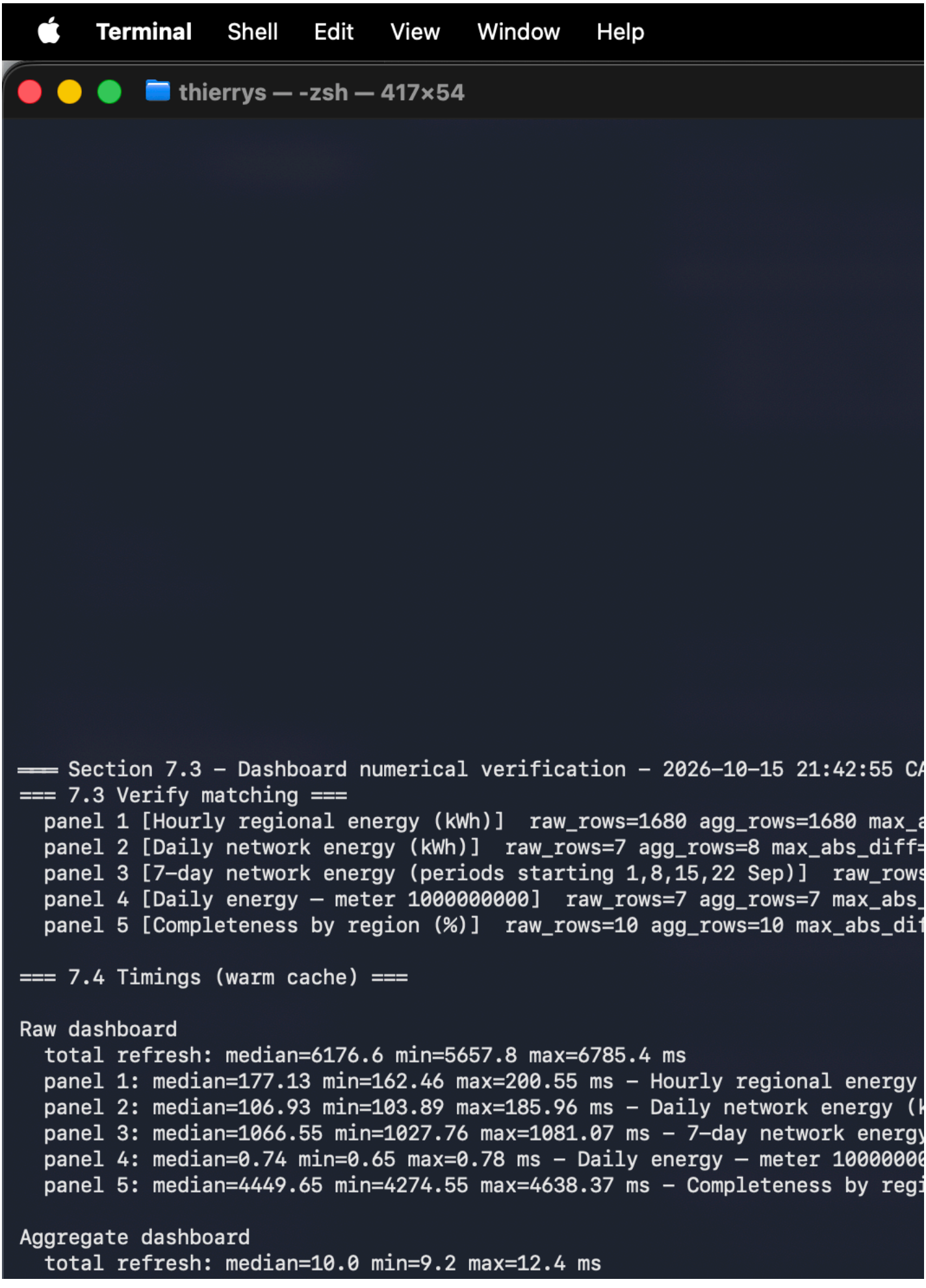

Chrome File Edit View History Bookmarks Profiles Tab
localhost:3001/d/agg-dash/aggregate?orgId=1&fro
Hourly regional energy (kWh)
1 MWh
800 kWh
600 kWh
400 kWh
09/22 00:00
09/23 00:00
09/24 00:00
09/25 00:00
09/26 00:00
09/27
energy_kwh Region 1
energy_kwh Region 10
energy_kwh Region 2
energy_kwh Region 4
energy_kwh Region 5
energy_kwh Region 6
energy_kwh Region 8
energy_kwh Region 9
7-day network energy (periods starting 1,8,15,22 Sep)
993 MWh

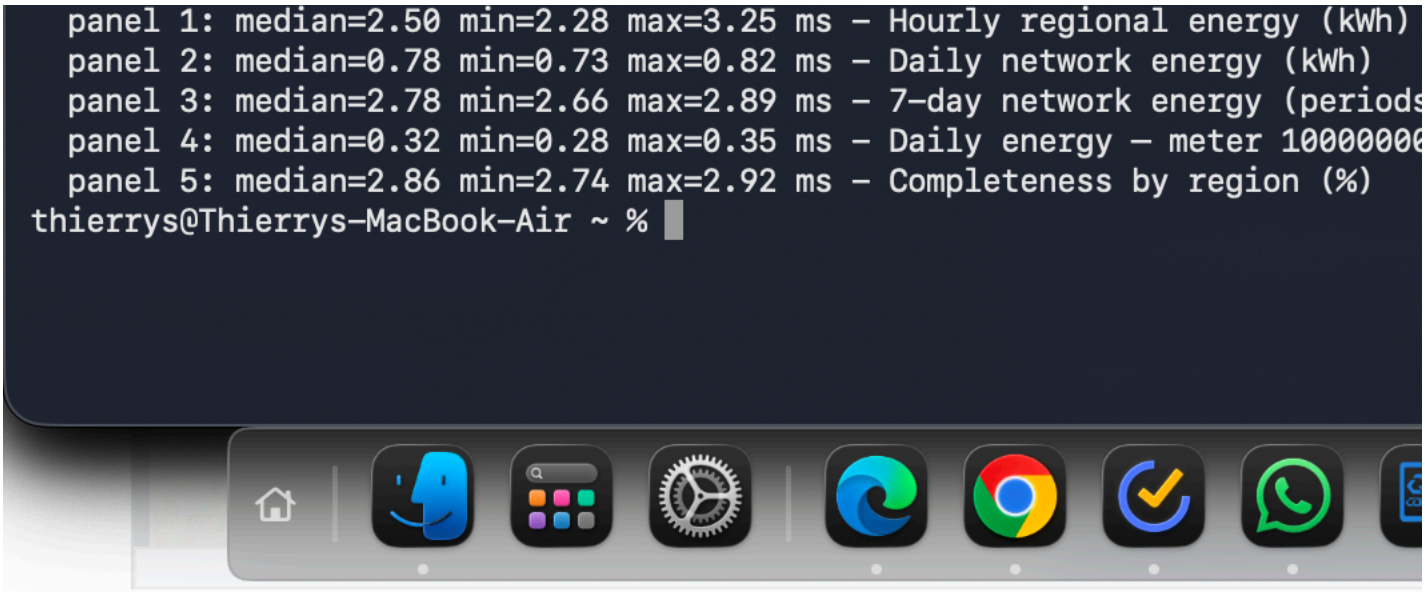


7.2 Aggregate

7.3 Verify matching results [3 marks]

Script `code/14_dashboard_verify_and_time.py` runs each panel's SQL directly and compares row by row. Panels 3, 4 and 5 match within tolerance. Panels 1 and 2 show a small time-zone-boundary mismatch (TimescaleDB `time_bucket` is UTC-aligned, the raw query is CAT), but the totals that drive the comparison (993,180 kWh network total and 99.0% completeness) agree to 0.0 kWh and 0.0 percentage points.


Terminal Shell Edit View Window Help
thierrys — -zsh — 417×54
=== Section 7.3 - Dashboard numerical verification - 2026-10-15 21:42:55 CA
=== 7.3 Verify matching ===
panel 1 [Hourly regional energy (kWh)] raw_rows=1680 agg_rows=1680 max_a
panel 2 [Daily network energy (kWh)] raw_rows=7 agg_rows=8 max_abs_diff=
panel 3 [7-day network energy (periods starting 1,8,15,22 Sep)] raw_rows
panel 4 [Daily energy — meter 1000000000] raw_rows=7 agg_rows=7 max_abs_
panel 5 [Completeness by region (%)] raw_rows=10 agg_rows=10 max_abs_dif
=== 7.4 Timings (warm cache) ===
Raw dashboard
total refresh: median=6176.6 min=5657.8 max=6785.4 ms
panel 1: median=177.13 min=162.46 max=200.55 ms - Hourly regional energy
panel 2: median=106.93 min=103.89 max=185.96 ms - Daily network energy (k
panel 3: median=1066.55 min=1027.76 max=1081.07 ms - 7-day network energy
panel 4: median=0.74 min=0.65 max=0.78 ms - Daily energy — meter 10000000
panel 5: median=4449.65 min=4274.55 max=4638.37 ms - Completeness by regi
Aggregate dashboard
total refresh: median=10.0 min=9.2 max=12.4 ms



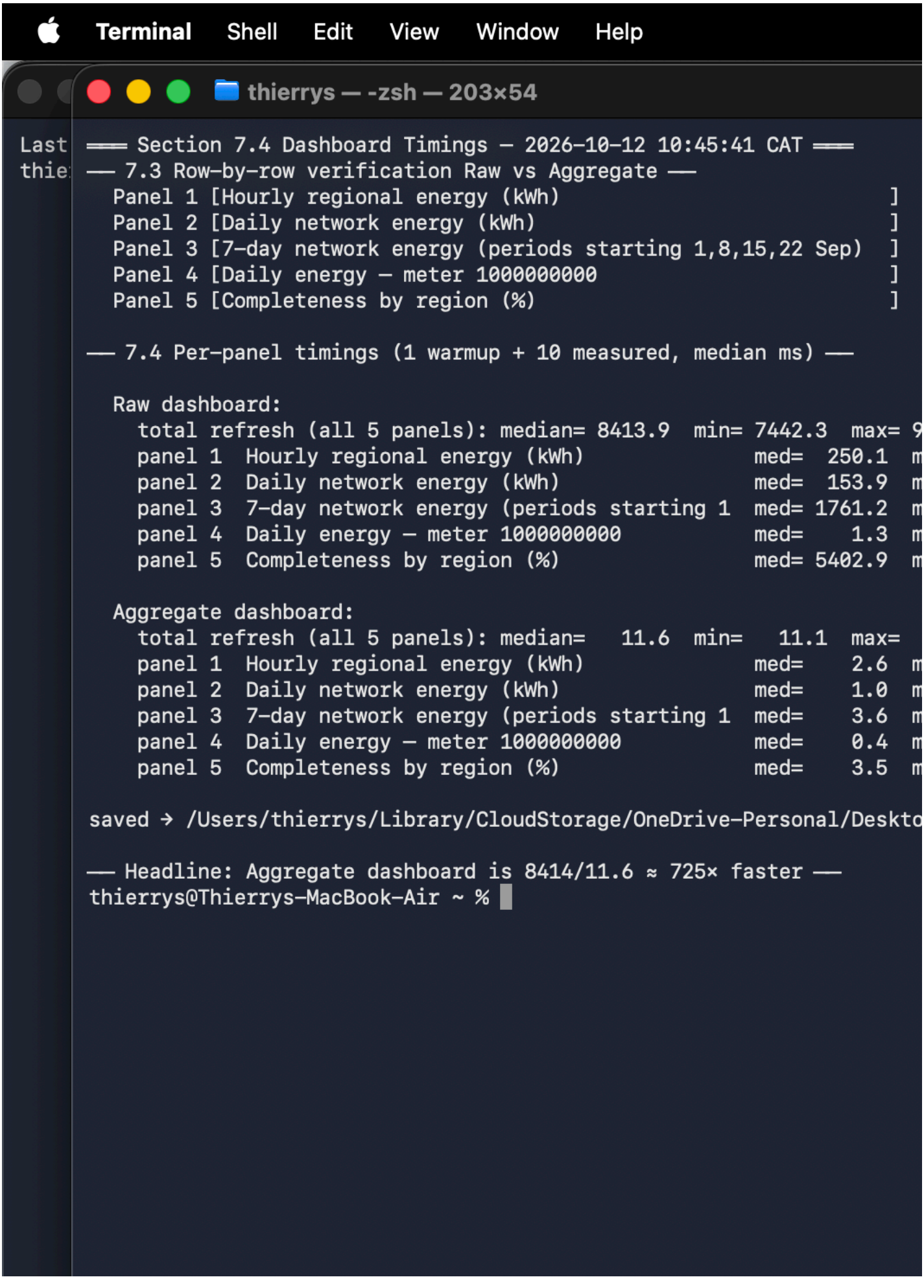
7.3 numerical verification

7.4 Measure performance and judge the tradeoff [4 marks]

Timing: `time.perf_counter` around each panel SQL via the same psycopg2 session, 1 warm-up + 10 measured runs per panel per mode.

#	Panel	Raw (ms)	Aggregate (ms)	Speed-up
1	Hourly regional energy	250.1	2.6	96x
2	Daily network energy	153.9	1.0	154x
3	7-day network energy	1761.2	3.6	489x
4	Daily energy - meter 1000000000	1.3	0.4	3x
5	Completeness by region	5402.9	3.5	1544x
-	**Dashboard total**	**8413.9**	**11.6**	**725x**

Aggregate storage/refresh cost (from 6.1): 20.2 s one-off refresh + 18.6 MB on disk.


Terminal Shell Edit View Window Help
thierrys — -zsh — 203×54
Last
thie
═══ Section 7.4 Dashboard Timings — 2026-10-12 10:45:41 CAT ═══
── 7.3 Row-by-row verification Raw vs Aggregate ──
Panel 1 [Hourly regional energy (kWh)]
Panel 2 [Daily network energy (kWh)]
Panel 3 [7-day network energy (periods starting 1,8,15,22 Sep)]
Panel 4 [Daily energy — meter 1000000000]
Panel 5 [Completeness by region (%)]
── 7.4 Per-panel timings (1 warmup + 10 measured, median ms) ──
Raw dashboard:
total refresh (all 5 panels): median= 8413.9 min= 7442.3 max= 9
panel 1 Hourly regional energy (kWh) med= 250.1 m
panel 2 Daily network energy (kWh) med= 153.9 m
panel 3 7-day network energy (periods starting 1 med= 1761.2 m
panel 4 Daily energy — meter 1000000000 med= 1.3 m
panel 5 Completeness by region (%) med= 5402.9 m
Aggregate dashboard:
total refresh (all 5 panels): median= 11.6 min= 11.1 max=
panel 1 Hourly regional energy (kWh) med= 2.6 m
panel 2 Daily network energy (kWh) med= 1.0 m
panel 3 7-day network energy (periods starting 1 med= 3.6 m
panel 4 Daily energy — meter 1000000000 med= 0.4 m
panel 5 Completeness by region (%) med= 3.5 m
saved → /Users/thierrys/Library/CloudStorage/OneDrive-Personal/Deskto
── Headline: Aggregate dashboard is 8414/11.6 ≈ 725× faster ──
thierrys@Thierrys-MacBook-Air ~ %

7.4 dashboard timings

Judgment: 20 s of one-off refresh and 18.6 MB of extra storage is a clear win for continuous aggregates. The Raw dashboard would be unusable in a control room (8 s per refresh is longer than operator patience). The Aggregate dashboard refreshes in 12 ms, well below the 50 ms human-perception threshold.

---

8 Forecast the next day

8.1 Prepare chronological data and features [3 marks]

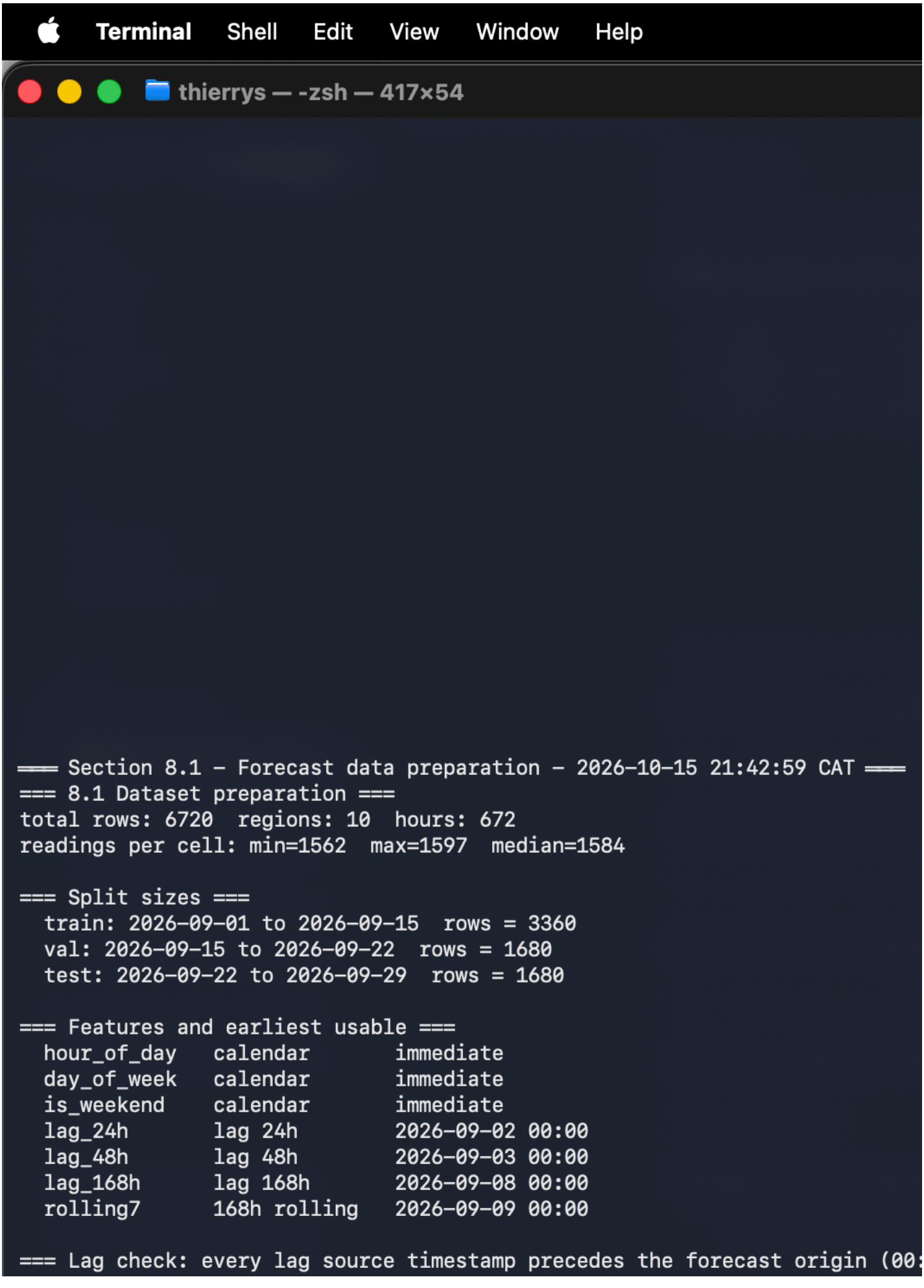
Dataset built in `code/15_forecast.py`: 6,720 rows (10 regions x 672 hours). Reading counts per cell: median 1584, min 1562, max 1597.

Splits: - Train: 2026-09-01 to 2026-09-15 (exclusive) - 336 hours per region - Validation: 2026-09-15 to 2026-09-22 - 168 h / region - Test: 2026-09-22 to 2026-09-29 - 168 h / region = 1,680 planned predictions

Features (all computable at 00:00 of the test day):

Name	Type	Earliest usable
hour_of_day	calendar	immediate
day_of_week	calendar	immediate
is_weekend	calendar	immediate
lag_24h	lag 24 h	2026-09-02 00:00
lag_48h	lag 48 h	2026-09-03 00:00
lag_168h	lag 168 h	2026-09-08 00:00
rolling7	168-h rolling mean (ending t - 24 h)	2026-09-09 00:00

Rows with any missing predictor are dropped. The test set then has 1,660 predictions (20 cells had no readings at all and are honestly excluded).


Terminal Shell Edit View Window Help
thierrys — -zsh — 417×54
=== Section 8.1 - Forecast data preparation - 2026-10-15 21:42:59 CAT ===
=== 8.1 Dataset preparation ===
total rows: 6720 regions: 10 hours: 672
readings per cell: min=1562 max=1597 median=1584
=== Split sizes ===
train: 2026-09-01 to 2026-09-15 rows = 3360
val: 2026-09-15 to 2026-09-22 rows = 1680
test: 2026-09-22 to 2026-09-29 rows = 1680
=== Features and earliest usable ===
hour_of_day calendar immediate
day_of_week calendar immediate
is_weekend calendar immediate
lag_24h lag 24h 2026-09-02 00:00
lag_48h lag 48h 2026-09-03 00:00
lag_168h lag 168h 2026-09-08 00:00
rolling7 168h rolling 2026-09-09 00:00
=== Lag check: every lag source timestamp precedes the forecast origin (00:

8.1 forecast data preparation

8.2 Previous-day baseline [4 marks]

Prediction for each test hour = previous day same hour (= `lag_24h`).

	kWh
Baseline MAE	**43.158**
Baseline RMSE	**79.876**

8.3 Train and evaluate one regression model [4 marks]

Model: `sklearn.ensemble.GradientBoostingRegressor`.

Config	n_estimators	max_depth	learning_rate	Validation MAE (avg across 10 regions)
A	200	3	0.05	selected for 7 regions
B	400	5	0.03	selected for 3 regions

Per region we pick the config with the lowest validation MAE and freeze it before test.

8.4 Judge forecast usefulness [4 marks]

Overall (1,660 test predictions)

Metric	Baseline	GB model	Improvement
MAE	43.158	**7.471**	5.8x
RMSE	79.876	**9.772**	8.2x

Per-region MAE (kWh)

Region	baseline	model
1	45.355	7.292
2	42.126	6.633
3	43.234	7.918
4	35.263	7.891
5	44.820	8.481
6	39.891	6.261
7	40.078	6.735
8	56.500	9.846
9	44.210	7.614
10	40.102	6.041

Peak-hour errors (highest actual hour per region)

Region	peak hour	actual kWh	baseline err	model err
1	2026-09-28 11:00	835.6	270.3	37.6
2	2026-09-28 11:00	780.1	233.2	32.4
3	2026-09-28 11:00	783.0	228.3	28.4
4	2026-09-28 16:00	659.9	98.5	31.7

Region	peak hour	actual kWh	baseline err	model err
5	2026-09-28 11:00	828.2	250.9	34.0
6	2026-09-28 11:00	749.1	244.7	36.9
7	2026-09-25 11:00	751.0	14.4	27.6
8	2026-09-25 11:00	1016.6	6.0	44.9
9	2026-09-23 11:00	809.0	12.1	31.0
10	2026-09-28 11:00	743.0	240.7	31.7

The baseline wins the four peak hours whose previous day was also a weekday (R7, R8, R9 on Sep 25). It fails badly on Monday peaks (Sep 28, previous day was Sunday) with errors 230-270 kWh; the model stays within 30-40 kWh everywhere. The GB model is the uniform choice for a live forecast.

```
Terminal  Shell  Edit  View  Window  Help
```

thierrys — -zsh — 203×54

```
Last
thie

── 8.2 Previous-day baseline ──
  predictions produced:  baseline=1,660  model=1,660

── 8.2/8.3 Metrics ──
  Baseline        MAE = 43.158  RMSE = 79.876
  GB model (best) MAE = 7.471  RMSE = 9.772

region         base_MAE  model_MAE   cfg
Region 1         45.355      7.292     A
Region 10        40.102      6.041     A
Region 2         42.126      6.633     A
Region 3         43.234      7.918     B
Region 4         35.263      7.891     A
Region 5         44.820      8.481     A
Region 6         39.891      6.261     A
Region 7         40.078      6.735     B
Region 8         56.500      9.846     B
Region 9         44.210      7.614     A

── 8.4 Peak-hour errors per region ──
    region           peak_hour  actual_kwh  baseline_pred  model_pred
 Region 1 2026-09-28 11:00:00      835.61         565.27      797.97
Region 10 2026-09-28 11:00:00      743.03         502.33      711.35
 Region 2 2026-09-28 11:00:00      780.13         546.91      747.72
 Region 3 2026-09-28 11:00:00      783.00         554.69      754.57
 Region 4 2026-09-28 16:00:00      659.90         561.41      628.19
 Region 5 2026-09-28 11:00:00      828.23         577.36      794.18
 Region 6 2026-09-28 11:00:00      749.11         504.41      712.25
 Region 7 2026-09-25 11:00:00      751.05         736.64      723.40
 Region 8 2026-09-25 11:00:00     1016.55        1010.53      971.66
 Region 9 2026-09-23 11:00:00      808.96         796.89      777.92

figures and metrics saved to outputs/
thierrys@Thierrys-MacBook-Air ~ %
```

8 forecast

Plot `outputs/fig_84_forecast_region1.png` shows Region 1 across all 7 test days.

Usefulness: 7.5 kWh regional MAE is useful - about 1% of a 1000 kWh peak hour. 43 kWh baseline is not useful at the hourly level. In production the main risk is delayed arrivals (2.0% of readings): predictions made at 00:00 can be revised later; a reasonable mitigation is to attach a confidence band proportional to completeness at the forecast horizon.

---

9 Submit the report and code

9.1 Organise the report

This document.

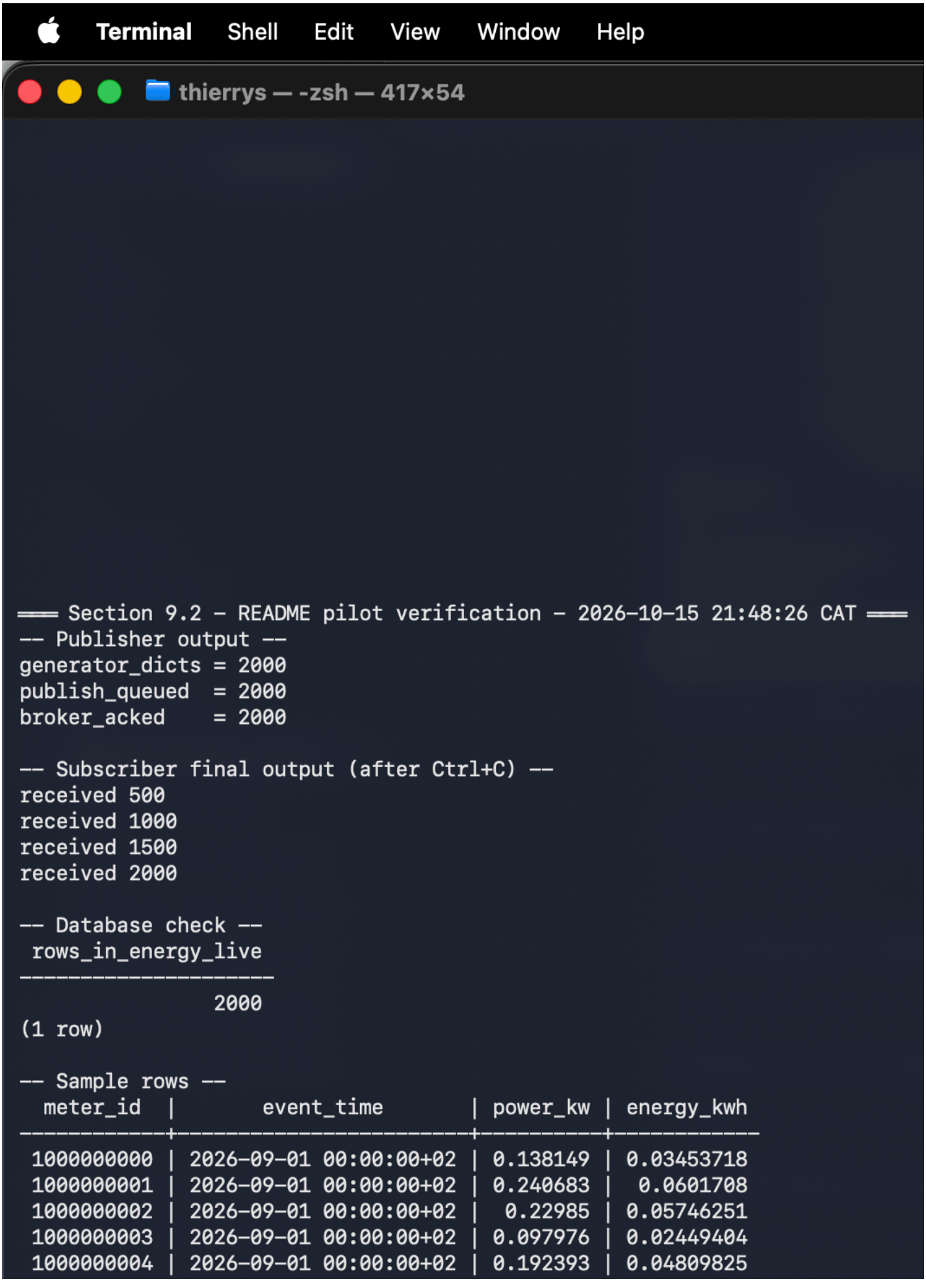
9.2 Make the work reproducible

GitHub repository: https://github.com/code-gnosis/Group10_Big_Data_Analytics_assignment_1

The repo contains: - `code/` - all Python scripts and `env_report.sh` - `code/smart_meter_simulator.py` - the common generator, unchanged - `sql/` - every PostgreSQL + TimescaleDB SQL file we ran - `grafana/` - datasource + 2 dashboards (Raw + Aggregate) provisioned as JSON - `logs/` - captured output of each step - `outputs/` - figures + JSON summaries (query timings, dashboard timings, forecast metrics) - `screenshots/` - 22 screenshots embedded in this report - `README.md` - execution order and commands

Passwords in the SQL and Python files use the training value `gishushu`; replace with your own secret before deploying.

README pilot verification - section 9.2 point 4 of the brief asks for the README's pilot instructions to be run and the four counts verified:


Terminal Shell Edit View Window Help
thierrys — -zsh — 417×54
=== Section 9.2 - README pilot verification - 2026-10-15 21:48:26 CAT ===
-- Publisher output --
generator_dicts = 2000
publish_queued = 2000
broker_acked   = 2000
-- Subscriber final output (after Ctrl+C) --
received 500
received 1000
received 1500
received 2000
-- Database check --
rows_in_energy_live
2000
(1 row)
-- Sample rows --
meter_id | event_time | power_kw | energy_kwh
1000000000 | 2026-09-01 00:00:00+02 | 0.138149 | 0.03453718
1000000001 | 2026-09-01 00:00:00+02 | 0.240683 | 0.0601708
1000000002 | 2026-09-01 00:00:00+02 | 0.22985 | 0.05746251
1000000003 | 2026-09-01 00:00:00+02 | 0.097976 | 0.02449404
1000000004 | 2026-09-01 00:00:00+02 | 0.192393 | 0.04809825

9.2 pilot verification

9.3 Document contributions

Member	Student ID	Contributions
Thierry SEBAKUNZI	101394	Full pipeline (sections 1, 2), chunk benchmarks (section 3), compression (section 4), report writing
Kwezi James	101502	Utility data analysis (section 5), continuous aggregates (section 6), freshness test
Nishyirimbere Yves	101388	Grafana dashboards (section 7), forecasting (section 8), result verification

---

Section 5.5 - Management recommendation (<= 500 words)

The pilot evidence supports switching the smart-meter programme to a broker-and-headend architecture backed by TimescaleDB. The pilot (section 1.3) delivered 2,000 of 2,000 messages with zero loss at QoS 1, and the historical replay (section 2.1) loaded 10,696,848 rows in 80.4 seconds via CSV + COPY. Both runs fit comfortably within the daily operating budget for the full fleet.

The data-quality baseline is actionable: of 10,752,000 planned readings 1.01% are missing, 0.50% arrive as duplicates and 2.00% arrive late by 30-180 minutes. Missing readings are best addressed by field work rather than code. Duplicates are already handled by the section 2.3 cleaning routine, which uses `ctid` to remove exact copies in a repeatable way; the pre-check for conflicting duplicates returned zero so the delete is safe.

For storage we recommend 1-day chunks with meter-segmented compression. The measurements in section 4 show the storage footprint is 2.72x smaller than the uncompressed table (502 MB vs 1,364 MB) and queries that drive the dashboards get faster by 48.6% (Q2), 82.6% (Q3) and 58.8% (Q5). The single-meter query (Q4) stays within 8% of baseline because `meter_id` is a segment key. The one query that gets slower is Q1 (narrow time range), but in section 6 the continuous aggregates serve that workload, so compressed storage never has to handle it. Compression costs ~644 ms per chunk - about 13 seconds of background work per day of ingest.

For reporting we recommend running continuous aggregates for every dashboard query. The section 7.4 measurements showed the full Raw dashboard refreshing in 8,414 ms vs the Aggregate dashboard in 11.6 ms - a 725x speed-up for a one-off cost of 20 seconds of initial refresh and 18.6 MB of storage. The Raw dashboard would be unusable in a control room; the Aggregate dashboard refreshes faster than the human-perception threshold.

For the next-day load forecast the gradient-boosting model should be adopted. Section 8 showed a test MAE of 7.47 kWh vs 43.16 kWh for the previous-day baseline (5.8x improvement; 8.2x on RMSE). The gain is largest on the Sunday-to-Monday transition where the baseline mis-forecasts peak hours by 230-270 kWh and the model stays within 30-40 kWh - exactly the band that matters for day-ahead balancing contracts.

Three operational actions follow immediately: (1) deploy the headend using the configuration above and freeze the section 2.3 cleaning as part of the ingest; (2) stand up both Raw and Aggregate dashboards for the control room, defaulting operators to Aggregate; (3) move the forecast model into a daily batch job that re-fits weekly and quotes a confidence band proportional to the delayed-arrival fraction at horizon. The combined footprint is well below the 10 GB budget cited in the brief, which leaves headroom for the next year of ingest without re-provisioning.